



TECH I CAN

Kairo

Curious today, Confident tomorrow

A practical beginner guide to tiny language models

Classroom Edition (2026)

Tech I Can Series

By Paul McMurray

Build. Train. Compare. Explain.

Tech I Can Kairo

Curious today, Confident tomorrow

Kairo: A Beginner-Friendly Guide to Building and Understanding Tiny Language
Models

Paul McMurray

Classroom Edition (2026)



Dedicated to all of the budding techies of the future.
The world is ready for your brilliance.

Copyright

Copyright © 2026 Paul McMurray. This project is distributed under the MIT License. Permission is granted to use, copy, modify, merge, publish, distribute, sublicense, and sell copies, subject to inclusion of the copyright and license notice.

Published by Tech I Can, for practical AI literacy in schools and workshops. See the repository LICENSE file for full terms and warranty disclaimer.

Publication Data

Publisher: Tech I Can

Publication date: May 2026

ISBN: Not assigned (Edition 1 school release, not for retail sale)

Edition: Classroom Edition (2026)

Trim size: A4 digital classroom edition

Tech I Can

Curious today, Confident tomorrow.

Printed in digital format for classroom distribution.

For classroom and school use, the Disclaimer on the following page applies alongside the licence terms above.

Disclaimer

This book is an educational guide for classroom and self-study use. It is not legal, safeguarding, or professional policy advice.

Kairo is a small local teaching model. Outputs can be incorrect, incomplete, biased, or unexpected. Always review outputs critically and use teacher supervision for classroom activity.

Before delivery, confirm your school or organisation IT rules, data policy, and age-appropriate usage standards. Do not use personal or sensitive student data in prompts, datasets, or saved logs.

All examples are provided as learning demonstrations. You remain responsible for local compliance and safe use.

About the Author

Teaching intent: The teaching intent is simple: clear steps, honest evidence, and confidence through practice.

I'm Paul McMurray, a software engineer, problem-solver, and lifelong learner from the North East of England.

My work has taken me deep into the world of technology, reliability, automation, and complex systems, but the part I enjoy most is making difficult things easier to understand. Whether I'm building tools, improving systems, writing guidance, or helping others find a clearer way forward, I'm drawn to practical ideas that make a real difference.

Outside of work, I'm a dad, a music lover, and someone who believes in staying curious. I enjoy soul, funk, and disco, long walks, good food, and finding new ways to teach my son how the world works - from computers and games to kindness, resilience, and asking good questions.

This book is personal to me because it reflects how I think: take something complicated, break it down, make it useful, and leave people with something they can actually do.

I don't pretend to have all the answers, but I've learned that progress often starts with asking better questions, being honest about what you don't know, and being willing to keep going. That spirit is what I've tried to bring into these pages.

Icons Used in This Book

You will see these symbols throughout the chapters. They signal what kind of guidance is being offered.

	Lightbulb Takeaway	A practical classroom takeaway you should remember.
	Definition	A clear meaning for a technical term used in the chapter.
	Note	Useful context, guardrails, or teaching detail for this step.
	Teacher Note	Classroom-delivery guidance intended for facilitators.
	Snippet Purpose	Why this command is being run before you use it.
	Snippet Change	What changed from a previous command and why it matters.

Contents

Use this grouped view to move quickly between setup, classroom workflow, advanced analysis, and reference pages.

SETUP

• Welcome to Tech I Can	15
• What Kairo Is (and Is Not)	19
• Before You Start	23
• Install and Verify	26

CORE WORKFLOW

• Run Kairo End-to-End	30
• How to Read Results	34
• Question-Answer Mode	37
• Learn Mode	41
• Classroom Delivery Plan	45
• Troubleshooting	49
• Safety and Honest Framing	52
• Build Your Own Mini Projects	55
• Full Command Reference	58
• Reflection Checkpoint	62

ADVANCED

• Dataset Design for Better Training	65
• Prompt Design for Fair Comparisons	68
• Tokenization Deep Dive	71
• Training Dynamics and Curves	74
• Evaluation Beyond One Number	78
• Reliability Patterns in Tiny Models	82
• Classroom Safety Workflow	85
• QA System Design Principles	88
• Guided Lab 1 (Baseline Build)	91
• Guided Lab 2 (Retrain Contrast)	95
• Guided Lab 3 (QA Grounding)	99

• Guided Lab 4 (Learn Mode Evidence Walkthrough)	103
--	-----

LABS

• Classroom Assessment and Feedback	106
• Presentation-Day Playbook	110
• Extension Pathways	113
• Advanced Prompt Engineering Lab	116
• Data Ethics and Attribution	120
• Full Classroom Script Pack	124
• Student Workbook Section	129
• Capstone Project Handbook	133

CLASSROOM OPS

• Implementation Workbook (Week-by-Week)	137
• Frequently Asked Questions (Classroom Edition)	142
• Teacher Quick-Reference Cards	147
• Deployment and Operations Checklist	152
• Accessibility and Inclusive Teaching Patterns	156
• Continuous Improvement Plan	160

REFERENCE

• Glossary (Terms and Parameters)	164
• Conclusion	169
• Key Words Index	173

Preface

Welcome to Tech I Can: Kairo. This book was written to make AI practical, teachable, and understandable for real classrooms.

You do not need prior AI expertise to use this guide well. You need a clear routine, honest observations, and a willingness to test one small change at a time. Every chapter is designed to help learners move from "I ran it" to "I can explain it."

Use this book to build a repeatable habit: explain why it exists, approach it with confidence, and understand what kind of learning journey to expect. That sequence will help you connect hands-on steps to clear reasoning.

This method also aligns with inquiry-based teaching: learners act, observe, and reflect before making broader claims. You can map the cycle in this book to well-known learning models where concrete experience and reflection build deeper understanding over time (Kolb, 1984; Hmelo-Silver, 2004).

In practical terms, that means this is not a "read once" book. It is a run-observe-discuss workbook. The goal is not perfect first attempts. The goal is steady reasoning quality that improves each time you test, compare, and explain what happened.

 **Lightbulb Takeaway: Confidence grows when you can explain one real change at a time, in your own words.**

This guide is intentionally written for people who are curious but may be new to programming or AI. You do not need to be an expert to begin. You only need a clear process, honest outputs, and a willingness to test your assumptions.

Who This Book Is For

Who Should Pick Up This Book

This page helps you decide quickly whether this book is the right fit for your current goals. It is written for people who want practical AI literacy, not hype, and who learn best by running real steps and discussing real outputs.

If you are teaching, this book gives you a classroom-ready structure with scripts, reflection prompts, and safety language. If you are learning independently, it gives you a clear pathway from first setup to confident explanation.

Target Audience

- secondary school teachers and learners
- FE/college educators introducing AI literacy
- workshop leaders and STEM clubs
- curious adult beginners who want practical understanding

Why This Book

Choose this book if you want to move from "AI sounds interesting" to "I can run it, inspect it, and explain it clearly." The structure is practical, repeatable, and designed for classrooms where time and confidence both matter.

Best Fit Check

- You want hands-on AI literacy instead of theory-only reading.
- You want guided commands with clear explanations and reflection.
- You want classroom-safe language that avoids overclaiming.

If This Book Is Not the Best First Step

If you want a deep mathematics-first treatment of transformer internals, start with a dedicated academic text. This book is a hands-on classroom and self-study guide focused on observable behavior, comparison, and explanation.

 **Lightbulb Takeaway:** This book is for learners and teachers who want practical confidence they can demonstrate.

How to Use This Book

Start Here

This quick-start page explains how to navigate the book so each chapter feels manageable and purposeful.

The structure is deliberate: orientation first, practical work second, then evidence-led reflection. That sequence is what turns command use into real understanding, especially for beginners.

Use this section as your route map. You will choose a pace, keep simple evidence notes, and build a study routine you can sustain across the full book.

Choose a Pacing Path

1. choose your pacing path (fast, standard, extended)
2. run one chapter at a time with notes and saved outputs
3. revisit glossary whenever a term is unclear

 **Note: Every chapter follows the same structure so learners always know what is coming next.**

 **Lightbulb Takeaway: Consistent workflow reduces overwhelm and improves confidence.**

 **Note: Kairo is a tiny, local, byte-level teaching model. Some patterns in this book, such as metric ranges, temperature effects, and failure modes, are specific to this setup and may look different in larger subword models trained on internet-scale datasets.**

Accessibility and Teaching Tips

- Read code blocks aloud in short chunks, then pause for paraphrase.
- Pair every command with one plain-language purpose before running it.
- Use captions and alt text as discussion prompts, not decoration.
- Offer two response options after each activity: spoken explanation or written claim-evidence note.
- Keep one "slow lane" and one "fast lane" option in each live session so mixed-confidence groups can stay together.



CHAPTER

Welcome to Tech I Can

1

In this chapter, you will explore the learning method used throughout the book, how chapters are structured, and how to read output with confidence. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- the learning method used throughout the book
- how chapters are structured
- how to read output with confidence

Foundations: Build confidence with the core workflow.

Chapter 1: Welcome to Tech I Can

About this chapter

You are about to learn AI by doing. This chapter sets the tone so you know what you are building and why each step matters.

Instead of treating AI as a black box, you will treat it as a system you can observe and discuss. The goal is not to memorize terms. The goal is to build a repeatable way to reason about model behavior.

By the end of this chapter, you should be able to describe the learning method used throughout the book, explain how chapters are structured, and read output with confidence.

What you are going to use

- this guide
- your local Kairo project
- a notebook for observations

What you will learn in this chapter

- the learning method used throughout the book
- how chapters are structured
- how to read output with confidence

The work, clearly laid out

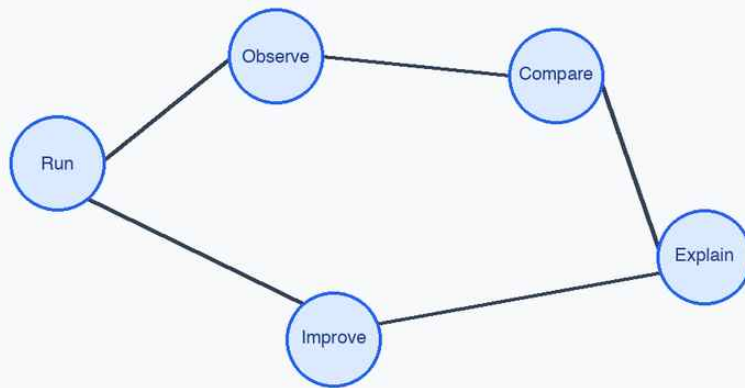
1. learn the five-step method
2. understand chapter flow
3. prepare for practical work

Examples of what you might see

■ Snippet Type: Text Output

```
Run -> Observe -> Compare -> Explain -> Improve
```

Learning Evidence Loop



Use this cycle for every chapter activity and reflection.

Alt text: Learning loop diagram showing five connected steps: Run, Observe, Compare, Explain, Improve.

Figure 1. The evidence loop used throughout the book. Keep your notes aligned to each step.

Why This Matters

Definition: In this book, Run means execute one step, Observe means capture what happened, Compare means check what changed, Explain means link claims to evidence, and Improve means choose one better next move.

Lightbulb Takeaway: You are not here to memorize commands. You are here to build an explanation habit.

Action 1: What You Learned

- You learned the five-step learning loop: run, observe, compare, explain, improve.
- You learned that this book rewards evidence, not guessing.
- You learned how to keep a simple record of what changed and why.

Action 2: Reflect

- Which step in Run -> Observe -> Compare -> Explain -> Improve gave you the clearest evidence in your notes?
- What exact note would you use to prove a claim in this chapter without guessing?
- Which note fields will you keep every time: command, output, change, explanation, and why?

Action 3: Do This Next

- Write one sentence for each step of the five-step loop using your own words.

- Create a one-page notes template with: command, output, change, explanation, then compare template choices with a peer.

CHAPTER

What Kairo Is (and Is Not)

2

In this chapter, you will explore what Kairo is designed for, what Kairo is not trying to be, and why tiny models are great for learning. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- what Kairo is designed for
- what Kairo is not trying to be
- why tiny models are great for learning

Foundations: Build confidence with the core workflow.

Chapter 2: What Kairo Is (and Is Not)

About this chapter

Before running code, you need realistic expectations.

By the end of this chapter, you should be able to explain what Kairo is designed for, explain what it is not trying to be, and describe why tiny models are useful for learning.

What you are going to use

- project overview
- sample dataset descriptions
- your own reasoning

What you will learn in this chapter

- what Kairo is designed for
- what Kairo is not trying to be
- why tiny models are great for learning

The work, clearly laid out

1. identify Kairo strengths
2. identify Kairo limits
3. connect both to classroom value

Examples of what you might see

■ Snippet Type: Text Output

Strong: clear style change after retraining

Limit: occasional repetitive text

Why This Matters

 **Definition:** Tiny model means a model intentionally small enough to train quickly and inspect locally.

 **Note:** Smaller models can fail more visibly, which is useful when teaching.

 **Note:** In this chapter, a "limit" means a case where Kairo is not meant to act like a production assistant, such as long-form factual reliability or broad open-domain knowledge.

 **Lightbulb Takeaway:** Honest limitations make better learning outcomes.

Action 1: What You Learned

- You learned what Kairo is built to do: show model behavior clearly in a small, teachable setup.
- You learned what Kairo is not built to do: act like a production-grade assistant.
- You learned why visible model limits are useful for real AI literacy.

Action 2: Reflect

- Which Kairo strength is most useful for your classroom and why?
- Which limit could confuse beginners if it is not explained early?
- How would you explain "small model, clear behavior" to a new learner?

Action 3: Do This Next

- Write two claims: one accurate claim about Kairo, one overclaim. Then correct the overclaim.
- Make a short "what this project can and cannot do" slide for learners, then compare it with a partner and agree one sentence to keep exactly the same in both versions.

Before You Book the Room: Your IT Pre-Flight Checklist

 **Note: Send this to your IT technician 2 weeks before delivery.**

- ☐ Python 3.11 or above is installed and accessible from the terminal
- ☐ pip package installer is available
- ☐ Terminal or command prompt access is permitted for student accounts
- ☐ `github.com` is accessible on the school network
- ☐ A local folder has read/write/execute permissions
- ☐ Python virtual environments can be created and activated
- ☐ (Optional) Docker is available for a containerised fallback

CHAPTER

Before You Start

3

In this chapter, you will explore required versus optional components, which extras unlock classroom features, and how to verify readiness. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- required versus optional components
- which extras unlock classroom features
- how to verify readiness

Foundations: Build confidence with the core workflow.

Chapter 3: Before You Start

About this chapter

This chapter prevents setup issues before they interrupt learning.

You will sort required tools from optional extras, choose the right install profile for your context, and confirm that your machine is ready before the first model run.

What you are going to use

- Python 3.11+
- terminal access
- local repository copy (<https://github.com/whiteyoh/my-llm>)

What you will learn in this chapter

- required versus optional components
- which extras unlock classroom features
- how to verify readiness

The work, clearly laid out

1. confirm Python version
2. confirm local repository
3. decide which extras to install

Examples of what you might see

■ Snippet Type: Text Output

Required: Python 3.11+

Optional extras: learn, pdf, dev

Why This Matters

📖 **Note:** Setup quality directly affects the quality of your first model run.

📖 **Note:** Minimum environment for this book is Python 3.11+, terminal access, and a local folder where you can install packages and run training commands. In school settings, ask your IT technician to confirm these permissions before the lesson week starts.

 **Definition:** A non-negotiable requirement is something that must work before any training command can succeed (Python version, terminal access, and repo files).

 **Lightbulb Takeaway:** A clean start removes most beginner friction.

Action 1: What You Learned

- You learned the difference between required tools and optional extras.
- You learned how early setup checks prevent later training failures.
- You learned which extras support classroom delivery and printable materials.

Action 2: Reflect

- Which requirement is non-negotiable before running any chapter commands?
- Which optional extra is most important for your current teaching goal?
- What is the first sign that your environment setup is incomplete?

Action 3: Do This Next

- Run a final preflight checklist and tick each item off by hand.
- Ask a peer to review your setup list and see if anything is missing.

CHAPTER

Install and Verify

4

In this chapter, you will explore how to install safely, how to verify readiness, and how to spot setup problems early. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to install safely
- how to verify readiness
- how to spot setup problems early

Foundations: Build confidence with the core workflow.

Chapter 4: Install and Verify

About this chapter

Now you turn this project into a working local AI lab.

This chapter moves you from preparation to action. You will install the project safely, verify the environment with quick checks, and learn early warning signs that prevent setup issues from growing during class.

What you are going to use

- virtual environment tools
- package installer
- quality checks

What you will learn in this chapter

- how to install safely
- how to verify readiness
- how to spot setup problems early

The work, clearly laid out

1. create environment
2. install project
3. run health checks

📄 **Snippet Purpose:** Create an isolated Python environment for this project.

🔄 **Snippet Change:** This creates and activates a project-specific Python runtime so package versions do not conflict with other projects.

■ **Snippet Type: Bash — Create an isolated Python environment for this project**

```
python -m venv .venv
source .venv/bin/activate
```

📄 **Snippet Purpose:** Install Kairo and optional classroom features.

🔄 **Snippet Change:** This installs command-line tools and optional Learn/PDF/dev extras used in later chapters.

■ Snippet Type: Bash — Install Kairo and optional classroom features

```
pip install -e .
pip install -e ".[learn]"
pip install -e ".[pdf]"
pip install -e ".[dev]"
```

🔗 **Snippet Purpose:** Confirm code quality, compilation health, and test stability.

🔄 **Snippet Change:** This validates that the environment and repository are healthy before model training.

■ Snippet Type: Bash — Confirm code quality, compilation health, and test stability

```
ruff check .
python -m compileall src tests tools
pytest -q
```

Examples of what you might see

■ Snippet Type: Text Output

```
All checks passed!
122 passed in 42.53s
```

Why This Matters

📖 **Definition:** Virtual environment (.venv) is a local Python space that keeps project dependencies isolated.

📝 **Note:** If one check fails, the recovery order is: read the first error line, fix the exact cause, then rerun only that check before running the full set.

💡 **Lightbulb Takeaway:** Verification is not extra work. It is what makes your next chapters trustworthy.

Action 1: What You Learned

- You learned how to isolate project dependencies with a virtual environment.
- You learned how to install core and optional features in a controlled order.
- You learned how lint, compile, and test checks confirm that the project is healthy.

Action 2: Reflect

- Why is a virtual environment safer than using global Python packages?

- Which verification command gives you the fastest signal when something is wrong?
- If one check fails, what is your next troubleshooting move?

Action 3: Do This Next

- Deactivate and reactivate the environment, then rerun the three health checks and compare the output.
- Ask a peer to run the same checks on their machine and compare whether any results differ.

CHAPTER

Run Kairo End-to-End

5

In this chapter, you will explore how to run a complete experiment, how to compare before/after behavior fairly, and why training data changes style. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to run a complete experiment
- how to compare before/after behavior fairly
- why training data changes style

Foundations: Build confidence with the core workflow.

Chapter 5: Run Kairo End-to-End

About this chapter

This chapter is your first complete model cycle: baseline, evaluation, retrain, comparison.

You will run one full baseline-to-retrain cycle, compare outputs under fixed conditions, and explain what changed with direct evidence from text and metrics.

What you are going to use

- data/samples/space_adventure.txt
- data/samples/pirate_dialogue.txt
- data/samples/README.md
- training, generation, and evaluation commands

What you will learn in this chapter

- how to run a complete experiment
- how to compare before/after behavior fairly
- why training data changes style

The work, clearly laid out

1. train baseline model
2. generate baseline output
3. evaluate baseline quality
4. retrain on contrast dataset
5. generate again with same prompt

 **Snippet Purpose:** Train a baseline checkpoint on neutral narrative text.

 **Snippet Change:** This creates your first reference checkpoint and establishes a baseline writing style.

■ Snippet Type: Bash — Train a baseline checkpoint on neutral narrative text

```
kairo-train --input_file data/samples/space_adventure.txt
--out_dir runs/book_demo_normal --epochs 1 --batch_size 4
--seq_len 32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose:** Generate baseline output from the saved checkpoint.

 **Snippet Change:** This produces the first output sample you will later compare after retraining.

■ **Snippet Type: Bash — Generate baseline output from the saved checkpoint**

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt
--prompt "Captain Rowan looked at the stars" --max_new_tokens 40
--temperature 0.9 --top_k 20 --device cpu
```

 **Snippet Purpose:** Measure baseline loss and perplexity on the same dataset.

 **Snippet Change:** This records baseline quality metrics for later before/after comparison.

■ **Snippet Type: Bash — Measure baseline loss and perplexity on the same dataset**

```
kairo-evaluate --checkpoint runs/book_demo_normal/best.pt
--input_file data/samples/space_adventure.txt --device cpu
```

 **Snippet Purpose:** Retrain the same architecture on pirate dialogue for contrast.

 **Snippet Change:** This shifts style behavior while keeping architecture fixed, enabling a fair dataset-effect test.

■ **Snippet Type: Bash — Retrain the same architecture on pirate dialogue for contrast**

```
kairo-train --input_file data/samples/pirate_dialogue.txt
--out_dir runs/book_demo_pirate --epochs 1 --batch_size 4
--seq_len 32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose:** Generate with the same prompt to make comparison fair.

 **Snippet Change:** This isolates the impact of retraining by keeping prompt and generation settings unchanged.

■ **Snippet Type: Bash — Generate with the same prompt to make comparison fair**

```
kairo-generate --checkpoint runs/book_demo_pirate/best.pt
--prompt "Captain Rowan looked at the stars" --max_new_tokens 40
--temperature 0.9 --top_k 20 --device cpu
```

Examples of what you might see

■ Snippet Type: Text Output

Baseline output: calm space narrative voice.

Retrained output: pirate vocabulary, exclamations, dialogue rhythm.

Why This Matters

📖 **Note:** Keep prompt and architecture fixed when comparing datasets.

📖 **Note:** In this experiment, the main variable is the dataset. Prompt, model settings, and architecture stay fixed so your comparison remains fair.

📖 **Note:** Record both output evidence and metrics (loss, perplexity) so your conclusion uses two evidence types.

📖 **Note:** With the chapter settings (--epochs 1, compact model, CPU), training typically takes about 2-5 minutes on a standard laptop. Slower machines may take longer, so plan one small timing buffer in class.

💡 **Lightbulb Takeaway:** Fair comparison means one major change at a time.

Action 1: What You Learned

- You learned how to run a complete baseline -> evaluate -> retrain -> compare cycle.
- You learned why keeping prompt and model settings fixed makes comparisons fair.
- You learned how dataset changes can shift vocabulary, tone, and rhythm.

Action 2: Reflect

- Which single variable changed between your baseline and retrained runs?
- What output evidence shows a style shift most clearly?
- Which metric did you record, and how did it support your conclusion?

Action 3: Do This Next

- Repeat the full cycle with a new fixed prompt and compare both result sets.
- Pair with a peer and compare one extra generation per checkpoint to see whether both of you reached the same style-shift conclusion.

CHAPTER

How to Read Results

6

In this chapter, you will explore how to assess style shifts, how to read metrics responsibly, and how to avoid overclaiming. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to assess style shifts
- how to read metrics responsibly
- how to avoid overclaiming

Foundations: Build confidence with the core workflow.

Chapter 6: How to Read Results

About this chapter

Now you move from running commands to interpreting evidence.

You will learn to read outputs and metrics together, so your conclusions stay clear, cautious, and evidence-led. By the end, you should be able to defend one claim with both textual examples and metric context, without drifting into overclaiming.

What you are going to use

- generated outputs
- loss/perplexity values
- comparison notes

What you will learn in this chapter

- how to assess style shifts
- how to read metrics responsibly
- how to avoid overclaiming

The work, clearly laid out

1. compare vocabulary and tone
2. compare repetition patterns
3. review metrics in context
4. write evidence-based conclusion

Examples of what you might see

■ Snippet Type: Text Output

Normal model: calmer narrative voice.

Pirate model: pirate terms and dramatic punctuation.

Why This Matters

 **Definition:** Perplexity is a measure of uncertainty; lower usually means better dataset fit, not better universal truth.

 **Note:** Perplexity can also fall when a model memorizes short, frequent patterns in small datasets. That is why you always inspect outputs alongside metrics.

 **Definition:** "Confident-sounding but wrong" means fluent language with weak or unsupported meaning. Treat fluency as style evidence, not truth evidence.

 **Lightbulb Takeaway:** Fluent output can still be wrong. Always separate style from reliability.

Action 1: What You Learned

- You learned to separate writing style changes from factual reliability.
- You learned how to read loss and perplexity as signals, not absolute truth.
- You learned how to make claims that are supported by visible output evidence.

Action 2: Reflect

- Which part of the output comparison is strongest evidence of style change?
- What can a lower perplexity score tell you, and what can it not tell you?
- Where could a confident-sounding output still be wrong?

Action 3: Do This Next

- Score two outputs with a simple rubric: tone, clarity, repetition, relevance.
- Swap rubric scores with a partner, then co-write one careful conclusion and one overclaim before revising the overclaim together.

CHAPTER

Question-Answer Mode

7

In this chapter, you will explore how QA corpora are built, how context anchors answers, and how fallback improves classroom reliability. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how QA corpora are built
- how context anchors answers
- how fallback improves classroom reliability

Foundations: Build confidence with the core workflow.

Chapter 7: Question-Answer Mode

About this chapter

This chapter changes behavior from open text generation to QA-style responses.

Use this chapter to build a repeatable habit: how QA corpora are built, how context anchors answers, and how fallback improves classroom reliability. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- `data/samples/qa_space_facts.jsonl`
- corpus-building command
- QA training and QA inference commands

What you will learn in this chapter

- how QA corpora are built
- how context anchors answers
- how fallback improves classroom reliability

The work, clearly laid out

1. build QA training corpus
2. train QA checkpoint
3. run a grounded QA query

📄 **Snippet Purpose:** Convert structured JSONL Q/A records into training text.

🔄 **Snippet Change:** This transforms question-answer records into model-trainable plain text format.

■ **Snippet Type: Bash — Convert structured JSONL Q/A records into training text**

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl  
--output_file runs/qa_space_facts.txt
```

📄 **Snippet Purpose:** Train a QA-focused checkpoint on the generated corpus.

🔄 **Snippet Change:** This creates a specialized checkpoint optimized for QA-style prompts.

■ Snippet Type: Bash — Train a QA-focused checkpoint on the generated corpus

```
kairo-train --input_file runs/qa_space_facts.txt --out_dir
runs/book_qa_demo --epochs 1 --batch_size 4 --seq_len 32
--d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

📖 Snippet Purpose: Ask a question with context so answers stay grounded.

🔄 Snippet Change: This tests grounded answering behavior using explicit context support.

■ Snippet Type: Bash — Ask a question with context so answers stay grounded

```
kairo-qa --checkpoint runs/book_qa_demo/best.pt --question "Who
pilots the Aurora?" --context "Captain Rowan is the pilot of the
starship Aurora."
```

Examples of what you might see

■ Snippet Type: Text Output

Answer: Captain Rowan is the pilot of the starship Aurora.

Why This Matters

📖 Definition: Grounding means constraining answers to supplied context.

📝 Note: Fallback behaviour is what happens when context is absent or too weak to anchor a response. In practice, Kairo may return a generic phrase, repeat part of the question, or produce lower-confidence output. This is expected behaviour, not a failure. Recognising fallback output is itself a classroom evidence point: it shows learners what grounding prevents.

💡 Lightbulb Takeaway: Context is your anchor when model confidence and correctness do not match.

Action 1: What You Learned

- You learned how JSONL question-answer data becomes trainable text.
- You learned how grounding (answering from provided context) improves answer stability.
- You learned why fallback behavior matters when context is missing or weak.

Action 2: Reflect

- Which part of your context sentence most improved answer precision?
- What output signs tell you the model answered beyond the provided context?
- Which fallback response should appear when context does not contain the answer?

Action 3: Do This Next

- Ask the same question with and without context, then compare answer reliability.
- Create one ambiguous question, swap it with a peer, and both rewrite it so grounding is easier; then compare the rewritten versions.

CHAPTER

Learn Mode

8

In this chapter, you will explore how tokenization appears visually, how next-token probabilities guide output, and how to inspect attention patterns carefully. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how tokenization appears visually
- how next-token probabilities guide output
- how to inspect attention patterns carefully

Foundations: Build confidence with the core workflow.

Chapter 8: Learn Mode

About this chapter

This chapter opens the model internals in a visual way.

You will not treat model visuals as decoration. You will use them as evidence: what tokens were read, which next-token choices were likely, and where the attention view can support discussion without overclaiming.

By the end of this chapter, you should be able to explain how tokenization appears visually, explain how next-token probabilities guide output, and inspect attention patterns carefully.

What you are going to use

- `kairo-learn`
- token preview view
- probabilities and attention views

What you will learn in this chapter

- how tokenization appears visually
- how next-token probabilities guide output
- how to inspect attention patterns carefully

The work, clearly laid out

1. launch Learn Mode
2. run a short training cycle
3. compare before/after outputs
4. inspect probabilities and attention

 **Snippet Purpose:** Start the Learn Mode app in your local browser.

■ **Snippet Type: Bash — Start the Learn Mode app in your local browser**

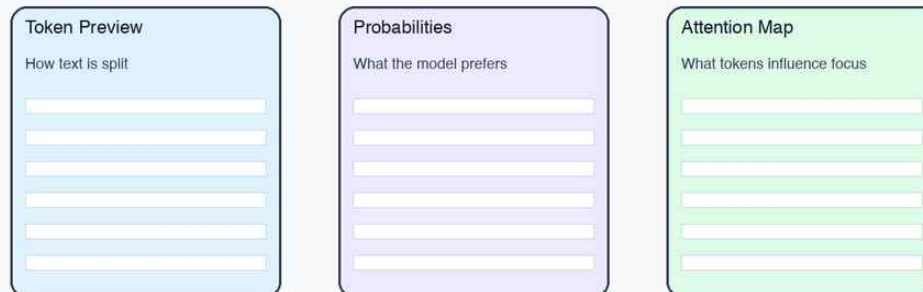
```
kairo-learn
```

Examples of what you might see

■ **Snippet Type: Text Output**

Launching Kairo Learn Mode...
Local URL: http://localhost:8501

Learn Mode Evidence Surfaces



Use each panel to support a specific claim in your reflection notes.

Alt text: Learn Mode visual concept showing token stream, probability focus, and attention map relationship.

Figure 2. Learn Mode is most useful when each visual is linked to a specific claim in your notes.

Why This Matters

 **Note:** Attention maps are interpretability aids, not proof of reasoning.

 **Note:** Token view shows the model's reading units, while plain output only shows the final text. Use both together when explaining behavior.

 **Lightbulb Takeaway:** Visual evidence helps learners connect abstract ideas to observable behavior.

Action 1: What You Learned

- You learned how token views show what the model actually reads.
- You learned how next-token probabilities shape output choices.
- You learned how attention visuals can help discussion, but do not prove reasoning.

Action 2: Reflect

- What did the token view reveal that plain text output did not?
- How did changing probability settings affect output variety?
- What is one useful insight from an attention view, and one thing it cannot prove?

Action 3: Do This Next

- Enter three short prompts with different punctuation and compare token splits.
- Compare annotated screenshots with a peer and discuss one insight and one limit for each view (tokens, probabilities, attention).

CHAPTER

Classroom Delivery Plan

9

In this chapter, you will explore how to run a 45-minute session, how to guide evidence discussion, and how to connect tasks to outcomes. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to run a 45-minute session
- how to guide evidence discussion
- how to connect tasks to outcomes

Classroom Core: Move from setup to repeatable delivery.

Chapter 9: Classroom Delivery Plan

About this chapter

Now you shift from learner mode to facilitator mode.

You will plan and run a complete 45-minute lesson with clear timing, strong evidence prompts, and a debrief that connects classroom activity to learning outcomes.

This chapter is about classroom choreography: what happens first, what must stay fixed, and where discussion should slow down so learners can reason from real outputs rather than quick guesses.

What you are going to use

- baseline/retrain workflow
- comparison prompt set
- structured reflection questions

What you will learn in this chapter

- how to run a 45-minute session
- how to guide evidence discussion
- how to connect tasks to outcomes

The work, clearly laid out

1. prediction warm-up
2. baseline run
3. retrain run
4. same-prompt comparison
5. reflection and debrief

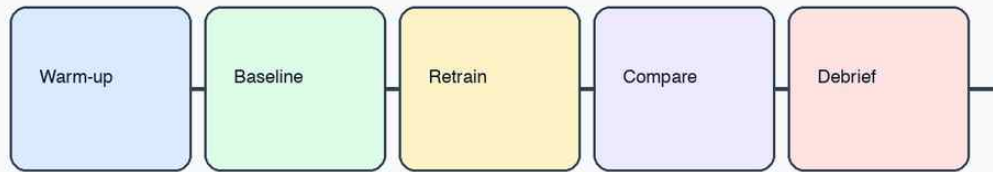
Examples of what you might see

■ Snippet Type: Text Output

"What changed, and what evidence supports that?"

"What stayed the same because architecture stayed the same?"

45-Minute Lesson Arc



Plan time intentionally so evidence discussion is never rushed.

Alt text: Lesson delivery map showing the five classroom phases from warm-up to debrief, with evidence prompts at each stage.

Figure 3. A full lesson arc is easier to run when each phase has one clear evidence goal.

Why This Matters

Note: The usual timing slip points are setup delays, long generation retries, and open-ended debriefs. Plan a "shorten first" option for each.

Note: A useful facilitation prompt is: "What exact output line supports your claim?" This moves learners from opinion language to evidence language.

Lightbulb Takeaway: The debrief is where understanding deepens.

Action 1: What You Learned

- You learned how to sequence a 45-minute lesson from prediction to debrief.
- You learned how to keep discussion anchored to outputs instead of opinions.
- You learned where to spend time in class so learners finish with a clear conclusion.

Action 2: Reflect

- Which part of your lesson needs the most facilitation: prediction, comparison, or debrief?
- What prompt will you use to move learners from "I think" to "I can show"?
- Where might timing slip, and what step will you shorten first?

Action 3: Do This Next

- Draft a minute-by-minute plan for one full classroom run.

- Rehearse the plan with a colleague and compare where each of you expects learner confusion.

CHAPTER

Troubleshooting

10

In this chapter, you will explore how to diagnose common issues, how to recover quickly, and how to keep learner confidence high. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to diagnose common issues
- how to recover quickly
- how to keep learner confidence high

Classroom Core: Move from setup to repeatable delivery.

Chapter 10: Troubleshooting

About this chapter

Every real workshop includes bumps. This chapter keeps progress moving.

By the end of this chapter, you should be able to diagnose common issues, recover quickly, and keep learner confidence high.

In live teaching, the quality of troubleshooting often decides whether learners feel capable or overwhelmed. This chapter gives you a simple triage mindset: name the failure clearly, test one small fix, and verify with a minimal rerun.

What you are going to use

- error messages
- run directory checks
- quick recovery settings

What you will learn in this chapter

- how to diagnose common issues
- how to recover quickly
- how to keep learner confidence high

The work, clearly laid out

1. classify issue type
2. apply targeted fix
3. rerun minimal command

Examples of what you might see

■ Snippet Type: Text Output

```
Error: checkpoint not found at runs/demo/best.pt
Tip: verify training completed and path is correct.
```

Why This Matters

📌 **Note:** Most issues are path, environment, or expectation mismatches.

 **Note:** Calm error language means naming the issue and next step clearly, for example: "The checkpoint path is missing; we will rerun training and verify."

 **Note:** A reliable triage order is reproduce -> isolate -> fix -> rerun -> record. If you skip the isolate step, you can apply a fix that appears to work once but fails again in the next session.

 **Lightbulb Takeaway:** Troubleshooting is not a detour. It is part of mastery.

Action 1: What You Learned

- You learned how to sort failures into path, environment, or expectation issues.
- You learned how to run a minimal rerun that confirms whether a fix worked.
- You learned how to keep learner focus when errors appear during live delivery.

Action 2: Reflect

- Which error type do you expect most often in your setup, and why?
- What is your fastest check when a checkpoint file cannot be found?
- How will you explain an error to learners without raising anxiety?

Action 3: Do This Next

- Create a quick "first five checks" card for your own machine.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

If Setup Cannot Be Completed

 **Note:** You can still deliver strong learning outcomes using the pre-generated outputs included in the repository.

 **Note:** Learners can read, annotate, and discuss real model outputs together without running commands locally.

 **Note:** A hosted or containerised fallback (Docker or Replit) may be available; check the project README for the latest route.

CHAPTER

Safety and Honest Framing

11

In this chapter, you will explore how to avoid overclaims, how to express uncertainty clearly, and how to model critical thinking. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to avoid overclaims
- how to express uncertainty clearly
- how to model critical thinking

Classroom Core: Move from setup to repeatable delivery.

Chapter 11: Safety and Honest Framing

About this chapter

This chapter helps you communicate results responsibly.

You will practice language that is honest and precise: avoid overclaiming, signal uncertainty clearly, and model critical thinking in front of learners.

You are not just choosing words here. You are setting classroom norms for how evidence is interpreted. Learners copy the language they hear from you, so your phrasing should separate observed output from assumptions about understanding.

What you are going to use

- your outputs
- metrics
- a language checklist for claims

What you will learn in this chapter

- how to avoid overclaims
- how to express uncertainty clearly
- how to model critical thinking

The work, clearly laid out

1. review output claims
2. check claim support
3. rewrite weak claims carefully

Examples of what you might see

■ Snippet Type: Text Output

Good claim: "Style shifted after pirate retraining."

Weak claim: "The model now understands pirates."

Why This Matters

 **Lightbulb Takeaway: Honest framing increases trust and learning quality.**

 **Note: Overclaim phrases to avoid include "the model understands" and "the model knows." Prefer "the output suggests" plus a cited line.**

 **Note: A useful claim ladder is: observed line -> interpretation -> confidence statement -> limitation. This keeps discussion honest without shutting down curiosity.**

If the model generates unexpected, offensive, or distressing output, pause the session immediately. Record the prompt and exact output, remove the content from display, and frame the moment as evidence of model limits and data influence. Then continue with a safer prompt and context pair.

Action 1: What You Learned

- You learned how to rewrite weak claims into evidence-based statements.
- You learned how to separate observed behavior from assumptions about understanding.
- You learned how careful language improves trust in classroom AI discussions.

Action 2: Reflect

- Which phrase in your own teaching could accidentally overclaim model ability?
- What evidence must be present before you call a result "reliable"?
- How will you model uncertainty when outputs are mixed?

Action 3: Do This Next

- Rewrite three model claims from your notes using safer, evidence-based wording.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Build Your Own Mini Projects

12

In this chapter, you will explore how to design controlled experiments, how to compare outputs rigorously, and how to write defensible conclusions. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to design controlled experiments
- how to compare outputs rigorously
- how to write defensible conclusions

Classroom Core: Move from setup to repeatable delivery.

Chapter 12: Build Your Own Mini Projects

About this chapter

This is where guided practice becomes independent experimentation.

Use this chapter to build a repeatable habit: design controlled experiments, compare outputs rigorously, and write defensible conclusions. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- two contrasting datasets
- one fixed prompt
- one fixed architecture

What you will learn in this chapter

- how to design controlled experiments
- how to compare outputs rigorously
- how to write defensible conclusions

The work, clearly laid out

1. choose datasets A and B
2. keep architecture fixed
3. keep prompt fixed
4. run both experiments
5. compare outputs and metrics
6. write conclusion with evidence

Examples of what you might see

■ Snippet Type: Text Output

Project: science explainers vs dramatic dialogue

Prompt: "The team opened the hatch"

Result: clear shift in tone and rhythm

Why This Matters

💡 **Lightbulb Takeaway:** Controlled experiments teach faster than complex mixed changes.

📋 **Note:** A fair mini project keeps one variable changing at a time and sets success criteria before running (for example: visible style shift plus one supporting metric trend).

Action 1: What You Learned

- You learned how to hold architecture and prompts fixed while testing dataset effects.
- You learned how to compare outputs and metrics as one argument, not separate notes.
- You learned how to turn an idea into a small experiment with defensible conclusions.

Action 2: Reflect

- Which variable must stay fixed in your project so the comparison remains fair?
- What evidence will you use if output style and metric signals disagree?
- How will you decide whether your mini project succeeded?

Action 3: Do This Next

- Design one mini project with a clear objective, fixed prompt set, and evidence rubric.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Full Command Reference

13

In this chapter, you will explore where each command fits in workflow, how to sequence commands correctly, and how to avoid command overload. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- where each command fits in workflow
- how to sequence commands correctly
- how to avoid command overload

Classroom Core: Move from setup to repeatable delivery.

Chapter 13: Full Command Reference

About this chapter

This chapter is your quick command toolkit for live use.

Use this chapter to build a repeatable habit: where each command fits in workflow, sequence commands correctly, and avoid command overload. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- core train/generate/evaluate commands
- QA commands
- interactive and printable commands

What you will learn in this chapter

- where each command fits in workflow
- how to sequence commands correctly
- how to avoid command overload

The work, clearly laid out

1. run core modeling commands
2. run QA commands
3. run tool and printable commands

 **Snippet Purpose: Train a compact baseline model.**

■ Snippet Type: Bash — Train a compact baseline model

```
kairo-train --input_file data/samples/space_adventure.txt  
--out_dir runs/demo --epochs 1 --batch_size 4 --seq_len 32  
--d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose: Generate output from the trained checkpoint.**

■ Snippet Type: Bash — Generate output from the trained checkpoint

```
kairo-generate --checkpoint runs/demo/best.pt --prompt "The  
robot opened the door" --max_new_tokens 20 --device cpu
```

 **Snippet Purpose:** Evaluate loss and perplexity on a chosen dataset.

■ **Snippet Type: Bash — Evaluate loss and perplexity on a chosen dataset**

```
kairo-evaluate --checkpoint runs/demo/best.pt --input_file
data/samples/space_adventure.txt --device cpu
```

 **Snippet Purpose:** Start a direct interactive chat loop.

■ **Snippet Type: Bash — Start a direct interactive chat loop**

```
kairo-chat --checkpoint runs/demo/best.pt --device cpu
```

 **Snippet Purpose:** Build a QA corpus text file from structured JSONL records.

■ **Snippet Type: Bash — Build a QA corpus text file from structured JSONL records**

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl
--output_file runs/qa_space_facts.txt
```

 **Snippet Purpose:** Ask a grounded QA question against a trained QA checkpoint.

■ **Snippet Type: Bash — Ask a grounded QA question against a trained QA checkpoint**

```
kairo-qa --checkpoint runs/qa_demo/best.pt --question "Who
pilots the Aurora?" --context "Captain Rowan is the pilot of the
starship Aurora."
```

 **Snippet Purpose:** Launch interactive and printable helper tools.

■ **Snippet Type: Bash — Launch interactive and printable helper tools**

```
kairo-learn
kairo-agents-dashboard
python tools/pdf/generate_printables.py
python tools/pdf/generate_tech_i_can_book.py
```

Examples of what you might see

■ **Snippet Type: Text Output**

```
Generated docs/printable/Tech_I_Can_Kairo_Book.pdf
```

Why This Matters

 **Lightbulb Takeaway:** Commands are tools for questions, not goals by themselves.

Action 1: What You Learned

- You learned how each command maps to a specific stage of the workflow.
- You learned how to avoid random command use by chaining commands with intent.
- You learned which utility commands support teaching, QA, and printable outputs.

Action 2: Reflect

- Which three commands form your minimum end-to-end teaching workflow?
- Which command is easiest to misuse without context, and how will you guard against that?
- When should you run utility tools instead of core model commands?

Action 3: Do This Next

- Build your own one-page command cheat sheet grouped by workflow stage.
- Compare your cheat sheet with a peer and merge into one classroom version.

CHAPTER

Reflection Checkpoint

14

In this chapter, you will explore how to consolidate what you have learned so far, how to carry the method into deeper chapters, and how to explain your current progress clearly to others. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to consolidate what you have learned so far
- how to carry the method into deeper chapters
- how to explain your current progress clearly to others

Classroom Core: Move from setup to repeatable delivery.

Chapter 14: Reflection Checkpoint

About this chapter

This chapter is a checkpoint where you pause, review evidence, and prepare for the deeper technical chapters ahead.

By the end of this chapter, you should be able to summarize what you have learned so far, carry the method into deeper chapters, and explain your current progress clearly to others.

What you are going to use

- your saved runs
- your comparison notes
- your next project idea

What you will learn in this chapter

- how to consolidate what you have learned so far
- how to carry the method into deeper chapters
- how to explain your current progress clearly to others

The work, clearly laid out

1. review strongest evidence examples
2. pick one mini-project
3. rerun the method independently

Examples of what you might see

■ Snippet Type: Text Output

```
Run -> Observe -> Compare -> Explain -> Improve
```

Why This Matters

💡 **Lightbulb Takeaway:** Confidence is built by repeated clear cycles, not one big perfect run.

📋 **Note:** A realistic one-week project usually means one objective, one dataset change, and one fixed prompt set you can rerun and explain.

Action 1: What You Learned

- You learned how to carry the run-observe-compare-explain-improve method into new topics.
- You learned how to choose a next project that is small enough to finish and evaluate.
- You learned how to keep momentum by reviewing saved evidence rather than relying on memory.

Action 2: Reflect

- Which part of the method will you keep exactly the same in your next project?
- What is one realistic project you can complete in one week?
- What evidence format will help you review progress after a month?

Action 3: Do This Next

- Write a one-week continuation plan with one concrete model experiment.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

Curious today, Confident tomorrow.

CHAPTER

Dataset Design for Better Training

15

In this chapter, you will explore how to choose data that teaches clearly, how to avoid mixed-style confusion, and how to build classroom-friendly corpora. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to choose data that teaches clearly
- how to avoid mixed-style confusion
- how to build classroom-friendly corpora

Classroom Core: Move from setup to repeatable delivery.

Chapter 15: Dataset Design for Better Training

About this chapter

You now move from using existing sample files to designing your own datasets with intent.

Use this chapter to build a repeatable habit: choose data that teaches clearly, avoid mixed-style confusion, and build classroom-friendly corpora. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- data/samples/README.md
- your own source text ideas
- a dataset design checklist

What you will learn in this chapter

- how to choose data that teaches clearly
- how to avoid mixed-style confusion
- how to build classroom-friendly corpora

The work, clearly laid out

1. pick one learning objective
2. choose text that matches that objective
3. remove noisy or unrelated sections
4. test one short training run
5. inspect whether output matches intent

Examples of what you might see

■ Snippet Type: Text Output

Objective: teach style transfer

Dataset A: calm narrative

Dataset B: dramatic dialogue

Result: visible vocabulary and rhythm shift

Why This Matters

 **Definition:** Dataset curation means choosing and shaping source text so training behavior is purposeful.

 **Note:** The best learning datasets are clean, focused, and interpretable.

 **Note:** A simple accept/reject rule can be: keep lines that match your objective, remove lines that mix unrelated tone, audience, or task type.

 **Lightbulb Takeaway:** If your dataset objective is unclear, your output analysis will also be unclear.

Action 1: What You Learned

- You learned how to align dataset content with a single teaching objective.
- You learned how noisy or mixed text weakens output interpretation.
- You learned how clean corpus boundaries make before/after comparisons clearer.

Action 2: Reflect

- Which lines in your source text support your objective, and which should be removed?
- How would mixed tone in one dataset distort your experiment?
- What simple rule will you use to accept or reject new source text?

Action 3: Do This Next

- Curate a 200-400 line sample dataset focused on one voice or task.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Prompt Design for Fair Comparisons

16

In this chapter, you will explore how to keep prompts comparable, how prompt wording changes output, and how to separate prompt effects from data effects. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to keep prompts comparable
- how prompt wording changes output
- how to separate prompt effects from data effects

Classroom Core: Move from setup to repeatable delivery.

Chapter 16: Prompt Design for Fair Comparisons

About this chapter

Prompt design controls the fairness of your experiment.

You will design fair prompt comparisons so you can tell whether changes came from prompt wording or from training data.

Many comparison errors come from silent prompt drift rather than model behavior. This chapter helps you lock prompt wording deliberately so your conclusions stay defensible when you present them to others.

What you are going to use

- one fixed prompt set
- your baseline and retrained checkpoints
- a prompt tracking table

What you will learn in this chapter

- how to keep prompts comparable
- how prompt wording changes output
- how to separate prompt effects from data effects

The work, clearly laid out

1. write 5 fixed prompts
2. run each prompt on baseline checkpoint
3. run same prompts on retrained checkpoint
4. compare output differences line by line

 **Snippet Purpose:** Use one prompt for simple baseline comparison.

■ Snippet Type: Bash — Use one prompt for simple baseline comparison

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt
--prompt "Captain Rowan looked at the stars" --max_new_tokens 40
--temperature 0.9 --top_k 20 --device cpu
```

 **Snippet Purpose:** Reuse the same prompt on the retrained model.

■ **Snippet Type: Bash — Reuse the same prompt on the retrained model**

```
kairo-generate --checkpoint runs/book_demo_pirate/best.pt
--prompt "Captain Rowan looked at the stars" --max_new_tokens 40
--temperature 0.9 --top_k 20 --device cpu
```

Examples of what you might see

■ **Snippet Type: Text Output**

Same prompt, different style cues:

- baseline: descriptive and calm
- retrained: pirate slang and emphatic punctuation

Why This Matters

 **Definition:** Controlled prompting means keeping prompt wording constant while changing only the condition being tested.

 **Note:** Your prompt tracking table should include prompt ID, exact wording, checkpoint name, generation settings, and one-line output summary. Without these fields, reproducibility usually breaks.

 **Lightbulb Takeaway:** Fixed prompts create fair evidence.

Action 1: What You Learned

- You learned how fixed prompts prevent accidental drift between model comparisons.
- You learned how small wording shifts can change output style and confidence.
- You learned how prompt tracking protects fairness when sharing results.

Action 2: Reflect

- Which prompt words are most likely to bias model tone in your tests?
- How would you prove that a change came from data, not prompt phrasing?
- What should your prompt tracking table include to support reproducibility?

Action 3: Do This Next

- Build a five-prompt comparison set with clear intent labels for each prompt.
- Run the same prompt set with a peer and compare where your interpretations differ.

CHAPTER

Tokenization Deep Dive

17

In this chapter, you will explore what byte-level tokenization does, why spacing and punctuation matter, and how token boundaries influence generation. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- what byte-level tokenization does
- why spacing and punctuation matter
- how token boundaries influence generation

Technical Depth: Understand why model behavior changes.

Chapter 17: Tokenization Deep Dive

About this chapter

This chapter explains how raw text becomes model-readable tokens.

Use this chapter to build a repeatable habit: understand what byte-level tokenization does, explain why spacing and punctuation matter, and how token boundaries influence generation. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- Learn Mode token preview
- short custom sentences
- punctuation-rich examples

What you will learn in this chapter

- what byte-level tokenization does
- why spacing and punctuation matter
- how token boundaries influence generation

The work, clearly laid out

1. open Learn Mode token preview
2. test simple and complex sentences
3. inspect how punctuation splits
4. compare token counts by sentence style

Examples of what you might see

■ Snippet Type: Text Output

Input: "Hello, class!"

Token pattern: H e l l o , [space] c l a s s !

Why This Matters

📖 **Definition:** Byte-level tokenization represents text as byte units rather than word pieces.

 **Note:** Tiny shifts in punctuation can change token sequence and model behavior.

 **Lightbulb Takeaway:** Tokens are the model's actual reading units.

Action 1: What You Learned

- You learned how byte-level tokenization breaks text into units the model can process.
- You learned how punctuation and spacing choices alter token boundaries and sequence length.
- You learned how token patterns influence the style and stability of generated output.

Action 2: Reflect

- Which sample sentence produced more tokens than you expected, and why?
- How did punctuation change token boundaries in your Learn Mode tests?
- Which tokenization pattern best explains one output behavior you observed?

Action 3: Do This Next

- Build a small table of five sentences with their token counts and boundary notes.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Training Dynamics and Curves

18

In this chapter, you will explore how loss usually changes per epoch, what overfitting can look like, and when to stop and inspect instead of continuing. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how loss usually changes per epoch
- what overfitting can look like
- when to stop and inspect instead of continuing

Technical Depth: Understand why model behavior changes.

Chapter 18: Training Dynamics and Curves

About this chapter

You now learn how to read training behavior over time.

This chapter teaches one of the most important AI literacy habits: never judge model quality from one number in isolation. You will compare patterns across epochs and decide when to continue, pause, or redesign the experiment.

In this chapter, you will review how loss usually changes per epoch, identify what overfitting can look like, and decide when to stop and inspect instead of continuing. Keep your notes evidence-based so you can explain not only what happened, but why it happened.

What you are going to use

- train/validation loss curves
- short and longer training runs
- comparison notes

What you will learn in this chapter

- how loss usually changes per epoch
- what overfitting can look like
- when to stop and inspect instead of continuing

The work, clearly laid out

1. run a short training session
2. record train and validation loss
3. run a slightly longer session
4. compare curve behavior

 **Snippet Purpose: Run a short training for curve observation.**

■ Snippet Type: Bash — Run a short training for curve observation

```
kairo-train --input_file data/samples/space_adventure.txt  
--out_dir runs/curve_short --epochs 1 --batch_size 4 --seq_len  
32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

📄 **Snippet Purpose:** Run a slightly longer training for comparison.

■ **Snippet Type: Bash — Run a slightly longer training for comparison**

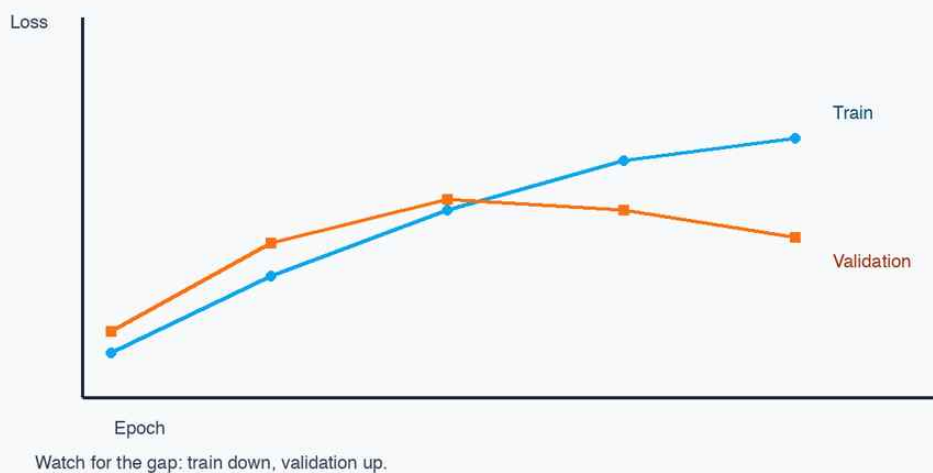
```
kairo-train --input_file data/samples/space_adventure.txt  
--out_dir runs/curve_long --epochs 3 --batch_size 4 --seq_len 32  
--d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

Examples of what you might see

■ **Snippet Type: Text Output**

```
Epoch 1: train_loss=4.20 | val_loss=4.08  
Epoch 2: train_loss=3.95 | val_loss=4.02  
Epoch 3: train_loss=3.76 | val_loss=4.10
```

Training vs Validation Curve Read



Alt text: Training chart with two lines where training loss falls steadily while validation loss flattens and then rises.

Figure 4. A widening train/validation gap is a practical signal to inspect for overfitting.

Why This Matters

📖 **Definition:** Overfitting is when training performance keeps improving while generalization quality stalls or worsens.

📝 **Note:** A single validation increase (for example 4.02 to 4.10) can be normal variance in tiny datasets. Treat one point as an inspection signal, not an automatic stop command. The case is stronger when the train/validation gap keeps widening across multiple epochs.

 **Note:** With these CPU-focused chapter settings, short runs are often around 2-5 minutes and longer runs around 5-12 minutes, depending on laptop speed and background load.

 **Lightbulb Takeaway:** Better training loss alone is not the full story.

Action 1: What You Learned

- You learned how to read train and validation loss together instead of focusing on one line.
- You learned how to spot early overfitting by watching validation stall while training keeps improving.
- You learned how to use short and longer runs (`runs/curve_short` and `runs/curve_long`) to make stop-or-continue decisions.

Action 2: Reflect

- At what point did validation stop improving in your longer run logs?
- Which curve pattern would make you stop training and inspect settings?
- What tradeoff did you notice between fewer epochs and output quality?

Action 3: Do This Next

- Run a two-epoch version of the same experiment and compare its curve shape with one and three epochs.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Evaluation Beyond One Number

19

In this chapter, you will explore how to combine metrics with text evidence, how to assess coherence and stability, and how to write balanced conclusions. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to combine metrics with text evidence
- how to assess coherence and stability
- how to write balanced conclusions

Technical Depth: Understand why model behavior changes.

Chapter 19: Evaluation Beyond One Number

About this chapter

This chapter expands evaluation from single metrics to richer interpretation.

Use this chapter to build a repeatable habit: combine metrics with text evidence, assess coherence and stability, and write balanced conclusions. That sequence will help you connect hands-on steps to clear reasoning.

What you are going to use

- evaluation outputs
- paired prompt generations
- a rubric for quality dimensions

What you will learn in this chapter

- how to combine metrics with text evidence
- how to assess coherence and stability
- how to write balanced conclusions

The work, clearly laid out

1. run `kairo-evaluate` on two checkpoints
2. generate 5 prompt outputs from each
3. score coherence, repetition, and relevance
4. summarize with metric + output evidence

 **Snippet Purpose: Evaluate baseline checkpoint on baseline dataset.**

■ Snippet Type: Bash — Evaluate baseline checkpoint on baseline dataset

```
kairo-evaluate --checkpoint runs/book_demo_normal/best.pt  
--input_file data/samples/space_adventure.txt --device cpu
```

 **Snippet Purpose: Evaluate retrained checkpoint on its dataset.**

■ Snippet Type: Bash — Evaluate retrained checkpoint on its dataset

```
kairo-evaluate --checkpoint runs/book_demo_pirate/best.pt
--input_file data/samples/pirate_dialogue.txt --device cpu
```

Examples of what you might see

■ Snippet Type: Text Output

Baseline: lower repetition, softer tone.

Retrained: stronger theme markers, occasional looping phrases.

Why This Matters

📝 **Note:** Metrics answer "how predictable," not "how meaningful."

📖 **Definition:** In this chapter, quality dimensions are coherence, repetition, and relevance; you score each one separately before writing conclusions.

📝 **Note:** Lower perplexity can come from memorized repetition in small corpora, not just stronger generalization. Pair metric changes with output inspection before making claims.

💡 **Lightbulb Takeaway:** Strong evaluation always combines numbers and language.

Action 1: What You Learned

- You learned how to combine `kairo-evaluate` metrics with direct text evidence from generated outputs.
- You learned how to score coherence, repetition, and relevance as separate quality dimensions.
- You learned how to write balanced evaluation statements when metrics and output quality do not fully agree.

Action 2: Reflect

- Which quality dimension changed most between baseline and retrained outputs?
- Which metric helped your conclusion, and where did text evidence matter more?
- How did you handle a case where a strong metric still produced weak language quality?

Action 3: Do This Next

- Create a one-page scoring sheet and evaluate two new prompts with the same rubric.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Reliability Patterns in Tiny Models

20

In this chapter, you will explore where tiny models are stable, where tiny models drift, and how settings influence reliability. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- where tiny models are stable
- where tiny models drift
- how settings influence reliability

Technical Depth: Understand why model behavior changes.

Chapter 20: Reliability Patterns in Tiny Models

About this chapter

This chapter helps you identify common reliability patterns.

You will test where tiny-model outputs stay stable, where they drift, and how generation settings influence that reliability profile. You will finish with a practical default setting profile for classroom demos and a short method for explaining why that profile is safer for beginners.

What you are going to use

- repeated prompt trials
- temperature variations
- context-grounded prompts

What you will learn in this chapter

- where tiny models are stable
- where tiny models drift
- how settings influence reliability

The work, clearly laid out

1. run same prompt 5 times
2. compare stability across outputs
3. lower and raise temperature
4. repeat with explicit context

 **Snippet Purpose:** Generate with moderate randomness.

■ Snippet Type: Bash — Generate with moderate randomness

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt
--prompt "The rescue crew entered the station" --max_new_tokens
50 --temperature 0.9 --top_k 20 --device cpu
```

 **Snippet Purpose:** Generate with lower randomness for stability check.

■ Snippet Type: Bash — Generate with lower randomness for stability check

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt
--prompt "The rescue crew entered the station" --max_new_tokens
50 --temperature 0.6 --top_k 20 --device cpu
```

Examples of what you might see

■ Snippet Type: Text Output

```
temperature=0.9 -> more variation and occasional odd phrases
temperature=0.6 -> more consistent but less expressive phrasing
```

Why This Matters

 **Definition:** Reliability here means consistency and groundedness across repeated runs under similar conditions.

 **Lightbulb Takeaway:** Settings can trade creativity for stability.

Action 1: What You Learned

- You learned how repeated prompt trials reveal whether a checkpoint is consistent or drifting.
- You learned how lowering temperature improved stability while reducing expressive variation.
- You learned how to select generation settings that are clearer and safer for classroom demonstrations.

Action 2: Reflect

- What reliability difference did you observe between temperature=0.9 and temperature=0.6?
- Which output traits became more stable when randomness was reduced?
- What setting choice would you use for a beginner-facing live demo, and why?

Action 3: Do This Next

- Run the same prompt five times at one additional temperature and log variation patterns.
- Compare your reliability logs with a peer and agree on a default classroom setting profile.

CHAPTER

Classroom Safety Workflow

21

In this chapter, you will explore how to prevent unsafe overclaiming, how to model responsible critique, and how to frame uncertainty constructively. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to prevent unsafe overclaiming
- how to model responsible critique
- how to frame uncertainty constructively

Technical Depth: Understand why model behavior changes.

Chapter 21: Classroom Safety Workflow

About this chapter

This chapter operationalizes safety for classroom demonstrations.

You will convert safety principles into repeatable classroom moves: set claim boundaries, question outputs responsibly, and frame uncertainty constructively. You will also practice the exact language you can use when an output is surprising, so discussion stays calm, accurate, and constructive.

What you are going to use

- prompt checklist
- context-first QA prompts
- review language for student discussions

What you will learn in this chapter

- how to prevent unsafe overclaiming
- how to model responsible critique
- how to frame uncertainty constructively

The work, clearly laid out

1. define claim boundaries before demo
2. use context for factual questions
3. highlight uncertainty language in output
4. debrief limits after each run

Examples of what you might see

■ Snippet Type: Text Output

"This output is plausible but not verified."

"Let's find supporting evidence before we accept this claim."

Why This Matters

 **Note:** Safety in this context includes epistemic safety: avoiding false certainty.

If an output is surprising or concerning, stop and reset the flow: capture the prompt/output pair, name why it is unsafe or unreliable, and relaunch with a safer prompt plus explicit context so students can compare the difference.

 **Note:** Learner responses that need careful facilitation include strong claims without evidence, overconfidence from fluent output, and frustration after drift.

 **Lightbulb Takeaway:** Responsible framing is a teaching skill, not a disclaimer.

Action 1: What You Learned

- You learned how to embed safety language into each stage of a live classroom workflow.
- You learned how to model critical questioning without dismissing learner curiosity.
- You learned how to frame uncertain outputs in ways that keep discussion calm and evidence-based.

Action 2: Reflect

- Which classroom sentence best signals uncertainty without shutting discussion down?
- Where in your lesson flow should you add a deliberate safety check?
- Which type of learner response needs the most careful facilitation during AI output review?

Action 3: Do This Next

- Write a short safety script with three lines you will use during live output review.
- Rehearse the script with a colleague and compare tone, clarity, and learner impact.

CHAPTER

QA System Design Principles

22

In this chapter, you will explore what makes QA training data effective, how context quality shapes answer quality, and how fallback behavior supports reliability. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- what makes QA training data effective
- how context quality shapes answer quality
- how fallback behavior supports reliability

Technical Depth: Understand why model behavior changes.

Chapter 22: QA System Design Principles

About this chapter

Now you treat QA mode as a system, not just a command.

You will treat QA as a designed system: improve corpus quality, strengthen context quality, and use fallback behaviour to keep answers dependable.

What you are going to use

- corpus build step
- QA checkpoint
- context quality checklist

What you will learn in this chapter

- what makes QA training data effective
- how context quality shapes answer quality
- how fallback behavior supports reliability

The work, clearly laid out

1. inspect QA record clarity
2. build corpus and train QA checkpoint
3. test varied question forms
4. compare context-rich vs context-poor behavior

Examples of what you might see

■ Snippet Type: Text Output

Context-rich question -> specific answer

Context-poor question -> generic or drifting answer

Why This Matters

 **Definition:** QA system quality depends on both model behavior and context design.

 **Note:** When context and question do not align well, the model may fall back to generic or less-specific output. This is the fallback behaviour referenced in the learning objectives. A useful classroom comparison is to run the same question with strong context and with no context, then observe where output drifts. The difference is the clearest demonstration of why context quality matters.

 **Lightbulb Takeaway:** Better context produces better answers more reliably than random parameter tweaking.

Action 1: What You Learned

- You learned how QA quality depends on the combined design of corpus, checkpoint, and context.
- You learned how context-rich prompts produce more specific and trustworthy answers than weak context.
- You learned how fallback behavior design improves reliability when question context is incomplete.

Action 2: Reflect

- Which context quality issue caused the weakest answer in your tests?
- What corpus design choice most improved QA answer consistency?
- Which fallback behavior should trigger when context and question do not align?

Action 3: Do This Next

- Build a context quality checklist and test it against three new QA prompts.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Guided Lab 1 (Baseline Build)**23**

In this chapter, you will explore how to execute a clean baseline lab, how to capture reproducible observations, and how to store outputs for later comparison. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to execute a clean baseline lab
- how to capture reproducible observations
- how to store outputs for later comparison

Technical Depth: Understand why model behavior changes.

Chapter 23: Guided Lab 1 (Baseline Build)

About this chapter

This lab is a full baseline run with checkpoint handling and evidence logging.

In this lab, you will execute a clean baseline run, capture reproducible notes, and store outputs in a form you can reuse for later comparisons.

What you are going to use

- `space_adventure.txt`
- training and generation commands
- a lab notes template

What you will learn in this chapter

- how to execute a clean baseline lab
- how to capture reproducible observations
- how to store outputs for later comparison

The work, clearly laid out

1. create run directory
2. train baseline
3. generate 3 outputs from fixed prompts
4. evaluate checkpoint
5. record findings

 **Snippet Purpose:** Train baseline model into a dedicated run folder.

■ Snippet Type: Bash — Train baseline model into a dedicated run folder

```
kairo-train --input_file data/samples/space_adventure.txt
--out_dir runs/lab1_baseline --epochs 1 --batch_size 4 --seq_len
32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose:** Generate output sample 1 for baseline notebook.

■ **Snippet Type: Bash — Generate output sample 1 for baseline notebook**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
"The mission clock started at dawn" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

📄 **Snippet Purpose: Generate output sample 2 for baseline notebook.**

■ **Snippet Type: Bash — Generate output sample 2 for baseline notebook**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
"The engineers checked the hull seals" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

📄 **Snippet Purpose: Generate output sample 3 for baseline notebook.**

■ **Snippet Type: Bash — Generate output sample 3 for baseline notebook**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
"Captain Rowan opened the flight log" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

📄 **Snippet Purpose: Evaluate baseline checkpoint and capture metrics.**

■ **Snippet Type: Bash — Evaluate baseline checkpoint and capture metrics**

```
kairo-evaluate --checkpoint runs/lab1_baseline/best.pt
--input_file data/samples/space_adventure.txt --device cpu
```

Examples of what you might see

■ **Snippet Type: Text Output**

Sample outputs are coherent but sometimes repetitive.
 loss: 4.0x
 perplexity: mid-50s

Why This Matters

📖 **Note: Baseline labs should be easy to repeat exactly.**

📖 **Note: Use one note format across the lab: prompt, output excerpt, metric, and one-sentence interpretation. This makes Lab 2 comparison clearer.**

💡 **Lightbulb Takeaway: Reproducibility is a core classroom skill.**

Action 1: What You Learned

- You learned how to run a clean baseline workflow from training to evaluation in `runs/lab1_baseline`.
- You learned how to document three fixed-prompt generations so later comparisons remain fair.
- You learned how to capture baseline metrics and notes that prepare a valid retrain contrast in Lab 2.

Action 2: Reflect

- Which of your three baseline prompts produced the most repeatable output pattern?
- What baseline metric will be most useful when you compare against Lab 2 results?
- Which note format made it easiest to connect output text with evaluation numbers?

Action 3: Do This Next

- Add one new fixed baseline prompt and record output with the same settings used in this lab.
- Compare baseline logs with a peer and identify one documentation improvement you both will adopt.

CHAPTER

Guided Lab 2 (Retrain Contrast)**24**

In this chapter, you will explore how retraining shifts style, how to preserve fair comparisons, and how to document changes with evidence. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how retraining shifts style
- how to preserve fair comparisons
- how to document changes with evidence

Technical Depth: Understand why model behavior changes.

Chapter 24: Guided Lab 2 (Retrain Contrast)

About this chapter

This lab demonstrates clear style transfer through retraining.

In this chapter, you will observe how retraining shifts style, preserve fair comparisons, and document changes with evidence. Keep your notes evidence-based so you can explain not only what happened, but why it happened.

What you are going to use

- pirate_dialogue.txt
- same architecture and prompts from Lab 1
- comparison worksheet

What you will learn in this chapter

- how retraining shifts style
- how to preserve fair comparisons
- how to document changes with evidence

The work, clearly laid out

1. retrain model with pirate data
2. rerun same 3 prompts
3. compare vocabulary, tone, and structure
4. write evidence-based summary

 **Snippet Purpose: Train contrast model with same architecture settings.**

■ Snippet Type: Bash — Train contrast model with same architecture settings

```
kairo-train --input_file data/samples/pirate_dialogue.txt
--out_dir runs/lab2_pirate --epochs 1 --batch_size 4 --seq_len
32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose: Re-run prompt 1 against retrained checkpoint.**

■ Snippet Type: Bash — Re-run prompt 1 against retrained checkpoint

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt
"The mission clock started at dawn" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

 **Snippet Purpose:** Re-run prompt 2 against retrained checkpoint.

■ **Snippet Type: Bash — Re-run prompt 2 against retrained checkpoint**

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt
"The engineers checked the hull seals" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

 **Snippet Purpose:** Re-run prompt 3 against retrained checkpoint.

■ **Snippet Type: Bash — Re-run prompt 3 against retrained checkpoint**

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt
"Captain Rowan opened the flight log" --max_new_tokens 50
--temperature 0.8 --top_k 20 --device cpu
```

Examples of what you might see

■ **Snippet Type: Text Output**

Pirate run uses "matey", "crew", "yo-ho", and dramatic punctuation.
Sentence rhythm shifts toward dialogue and commands.

Why This Matters

 **Definition:** Style transfer in this context means shifting generated language patterns through changed training data.

 **Note:** Lab 2 keeps the same prompts and settings from Lab 1 so any contrast is more likely due to retraining data, not prompt drift.

 **Lightbulb Takeaway:** Same model structure, different data, different behavior.

Action 1: What You Learned

- You learned how retraining on `pirate_dialogue.txt` changed language style while architecture stayed fixed.
- You learned how reusing the same three prompts made the baseline versus retrain comparison defensible.

- You learned how to explain style transfer using concrete evidence in vocabulary, tone, and sentence rhythm.

Action 2: Reflect

- Which specific words or punctuation patterns most clearly signaled style transfer?
- Which prompt showed the strongest contrast between Lab 1 and Lab 2 outputs?
- How did fixed settings protect your conclusion from accidental bias?

Action 3: Do This Next

- Write a short contrast summary that cites one output line from each of the three prompts.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Guided Lab 3 (QA Grounding)

25

In this chapter, you will explore how to prepare QA data, how to test grounded answers, and how to detect drift and recovery. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to prepare QA data
- how to test grounded answers
- how to detect drift and recovery

Technical Depth: Understand why model behavior changes.

Chapter 25: Guided Lab 3 (QA Grounding)

About this chapter

This lab turns the model into a basic context-grounded QA experience.

You will see that answer quality depends as much on context quality as on model weights. This is a key classroom insight: better prompts and better context often improve outcomes faster than bigger model settings.

By the end of this chapter, you should be able to prepare QA data, test grounded answers, and detect drift and recovery.

What you are going to use

- `qa_space_facts.jsonl`
- corpus builder
- QA command with context

What you will learn in this chapter

- how to prepare QA data
- how to test grounded answers
- how to detect drift and recovery

The work, clearly laid out

1. build QA corpus
2. train QA checkpoint
3. ask 5 factual questions
4. compare with and without context

 **Snippet Purpose:** Build QA corpus text from JSONL records.

■ Snippet Type: Bash — Build QA corpus text from JSONL records

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl
--output_file runs/lab3_qa_corpus.txt
```

 **Snippet Purpose:** Train QA checkpoint from lab corpus.

■ **Snippet Type: Bash — Train QA checkpoint from lab corpus**

```
kairo-train --input_file runs/lab3_qa_corpus.txt --out_dir
runs/lab3_qa --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
--n_heads 4 --n_layers 2 --device cpu
```

 **Snippet Purpose:** Ask a grounded question with explicit context.

■ **Snippet Type: Bash — Ask a grounded question with explicit context**

```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "Who
pilots the Aurora?" --context "Captain Rowan is the pilot of the
starship Aurora."
```

 **Snippet Purpose:** Ask a second grounded question for consistency check.

■ **Snippet Type: Bash — Ask a second grounded question for consistency check**

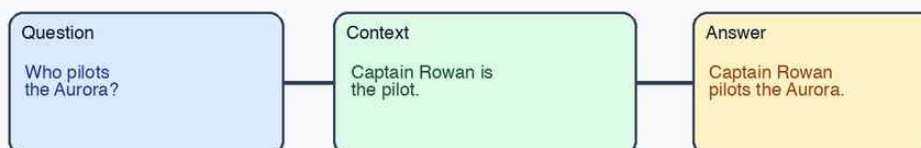
```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "What
powers the Aurora?" --context "The Aurora uses a helium-3 fusion
core."
```

Examples of what you might see

■ **Snippet Type: Text Output**

Context provided -> concise grounded answer.
Context missing -> broader, less anchored phrasing.

Grounded QA Flow



Better context gives clearer, safer answers.

Alt text: Grounded QA flow showing question input, context evidence, and constrained answer output.

Figure 5. Grounded QA works best when context contains direct, concise evidence for the question.

Why This Matters

 **Note:** QA quality improves most through context quality, not purely model size.

 **Note:** Drift clues include answers that ignore key context words, add unsupported facts, or switch to generic filler language.

 **Lightbulb Takeaway:** Good context design is the strongest lever in classroom QA.

Action 1: What You Learned

- You learned how to build and train a QA checkpoint from structured facts using a repeatable lab workflow.
- You learned how answer quality improved when context lines were specific and directly relevant to the question.
- You learned how to detect drift by comparing grounded and ungrounded responses across the same question set.

Action 2: Reflect

- Which question in your lab produced the clearest grounded answer, and what made the context effective?
- What output clue showed the model was drifting beyond the provided facts?
- Which context edit improved a weak answer the most?

Action 3: Do This Next

- Run five questions with context and five without context, then score groundedness for each answer.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Guided Lab 4 (Learn Mode Evidence Walkthrough)

26

In this chapter, you will explore how to read probability distributions, how to interpret attention cautiously, and how to narrate internals in plain language. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to read probability distributions
- how to interpret attention cautiously
- how to narrate internals in plain language

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 26: Guided Lab 4 (Learn Mode Evidence Walkthrough)

About this chapter

This lab helps students connect model internals to observable output behavior.

In this chapter, you will read probability distributions, then interpret attention cautiously, and finally narrate internals in plain language. Keep your notes evidence-based so you can explain not only what happened, but why it happened.

What you are going to use

- Learn Mode interface
- a fixed prompt list
- probability and attention views

What you will learn in this chapter

- how to read probability distributions
- how to interpret attention cautiously
- how to narrate internals in plain language

The work, clearly laid out

1. open Learn Mode
2. run one prompt in token view
3. inspect top next-token probabilities
4. inspect attention map for same example
5. write a two-sentence evidence summary

 **Snippet Purpose:** Launch Learn Mode app for visual exploration.

■ **Snippet Type:** Bash — Launch Learn Mode app for visual exploration

```
kairo-learn
```

Examples of what you might see

■ **Snippet Type:** Text Output

Top token candidates:

- 1) "the" 0.18
- 2) "and" 0.14
- 3) "to" 0.11

Why This Matters

 **Definition:** Next-token probabilities are candidate likelihoods before sampling.

 **Note:** Attention indicates influence patterns, not definitive reasoning intent.

 **Lightbulb Takeaway:** Visual evidence helps learners ask better questions.

Action 1: What You Learned

- You learned how to connect top-token probabilities to the actual words the model produced next.
- You learned how to discuss attention views as influence signals without claiming they prove full reasoning.
- You learned how to guide learners from visual observations to clear, evidence-based interpretations.

Action 2: Reflect

- Which probability shift best explained a surprising output choice in your walkthrough?
- What is one attention pattern you observed, and what is the safest conclusion you can draw from it?
- Which visual panel helped most when explaining model behavior to learners?

Action 3: Do This Next

- Capture one Learn Mode example and annotate tokens, probabilities, and attention in a single evidence sheet.
- Compare your annotated sheet with a peer and discuss one place where your interpretations differ.

CHAPTER

Classroom Assessment and Feedback

27

In this chapter, you will explore how to assess process and explanation quality, how to grade evidence use fairly, and how to provide actionable feedback. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to assess process and explanation quality
- how to grade evidence use fairly
- how to provide actionable feedback

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 27: Classroom Assessment and Feedback

About this chapter

This chapter provides concrete ways to assess learning outcomes.

By the end of this chapter, you should be able to assess process and explanation quality, grade evidence use fairly, and provide actionable feedback.

Strong assessment in this course does not reward speed alone. It rewards how well learners justify claims from outputs, metrics, and method choices. That is why the rubric emphasizes reasoning quality, not only command completion.

What you are going to use

- student lab notes
- output comparison artifacts
- reflection prompts and rubric criteria

What you will learn in this chapter

- how to assess process and explanation quality
- how to grade evidence use fairly
- how to provide actionable feedback

The work, clearly laid out

1. assess workflow completion
2. assess evidence quality
3. assess explanation clarity
4. provide next-step feedback

Examples of what you might see

■ Snippet Type: Text Output

Strong response:

"I kept the prompt fixed and changed only dataset. Output tone shifted from calm narrative to pirate dialogue markers."

Why This Matters

 **Definition:** Evidence quality means claims are supported by concrete output and workflow details.

 **Note:** The rubric categories used here are method quality, evidence quality, and explanation clarity. Use all three when comparing submissions.

Rubric descriptors

Method quality

- Strong: Prompt was fixed across runs; only one variable changed; commands are recorded in the correct sequence.
- Developing: Some controls were held but at least one was inconsistent; command sequence partially recorded.

Evidence quality

- Strong: Claim is supported by a specific output line or metric value; evidence is directly quoted or referenced.
- Developing: Claim is present but supported by general description rather than a specific output line or number.

Explanation clarity

- Strong: Explanation connects the evidence to the cause (e.g. "style shifted because the dataset changed, not the architecture").
- Developing: Explanation names what changed but does not connect it to the experimental design.

 **Note:** Feedback is most actionable when it names one strength, one gap, and one specific next experiment step. Generic praise or generic criticism rarely improves the next lab.

 **Lightbulb Takeaway:** Grade thinking quality, not just command execution.

Action 1: What You Learned

- You learned how to assess learner work by weighting evidence quality, method quality, and explanation clarity.

- You learned how to give feedback that improves reasoning, not just command execution speed.
- You learned how rubric-based assessment can guide the next experiment step for each learner.

Action 2: Reflect

- Which rubric category most clearly distinguished strong and weak submissions?
- What feedback sentence would help a learner improve evidence use in the next lab?
- Where did learners show understanding even when their command output was imperfect?

Action 3: Do This Next

- Score two sample responses using your rubric and write one concrete next-step comment for each.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Presentation-Day Playbook

28

In this chapter, you will explore how to prepare a smooth live session, how to handle surprises calmly, and how to keep outcomes teachable. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to prepare a smooth live session
- how to handle surprises calmly
- how to keep outcomes teachable

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 28: Presentation-Day Playbook

About this chapter

This chapter prepares you for live delivery in front of students or reviewers.

You will prepare for live delivery with a primary demo path, a fallback path, and transition language that keeps teaching value high even when issues appear.

What you are going to use

- one known-good checkpoint backup
- one scripted demo sequence
- one troubleshooting fallback plan

What you will learn in this chapter

- how to prepare a smooth live session
- how to handle surprises calmly
- how to keep outcomes teachable

The work, clearly laid out

1. rehearse complete demo once
2. pre-generate one backup output set
3. verify checkpoints and paths
4. pre-write key reflection questions

Examples of what you might see

■ Snippet Type: Text Output

Plan A: live train + generate

Plan B: load known-good checkpoint + run comparison prompts

Why This Matters

 **Note:** A calm fallback plan is a mark of professional readiness.

 **Note:** Pre-generated artifacts can include saved outputs, screenshots, and a short backup script order. Use these to keep flow when live runs fail.

 **Note:** If you switch plans, keep learners engaged by asking prediction questions before showing the backup artifact.

 **Lightbulb Takeaway:** Great presentations are prepared for both success and hiccups.

Action 1: What You Learned

- You learned how to prepare a live demo with a primary path and a ready fallback path.
- You learned how backup checkpoints and scripted transitions reduce disruption when technical issues appear.
- You learned how to preserve teaching value by keeping comparisons and reflection prompts ready under time pressure.

Action 2: Reflect

- Which part of your live sequence is most likely to fail, and what is your fallback move?
- What pre-generated artifact gives you the biggest recovery advantage during a demo?
- How will you keep learners engaged if you must switch from Plan A to Plan B?

Action 3: Do This Next

- Rehearse your full presentation once using only Plan B resources.
- Run a peer mock session where one person introduces a surprise failure and the other recovers live.

CHAPTER

Extension Pathways

29

In this chapter, you will explore how to design advanced follow-up projects, how to compare multiple domains, and how to turn this into ongoing coursework. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to design advanced follow-up projects
- how to compare multiple domains
- how to turn this into ongoing coursework

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 29: Extension Pathways

About this chapter

This chapter helps you keep growing after the core curriculum.

By the end of this chapter, you should be able to design advanced follow-up projects, compare multiple domains, and turn this into ongoing coursework.

What you are going to use

- your completed labs
- extension project ideas
- local documentation and guides

What you will learn in this chapter

- how to design advanced follow-up projects
- how to compare multiple domains
- how to turn this into ongoing coursework

The work, clearly laid out

1. select one extension theme
2. define measurable question
3. run controlled experiment
4. document findings and limits
5. present results to peers

Examples of what you might see

■ Snippet Type: Text Output

Extension themes:

- science vs poetry style transfer
- QA reliability by context quality
- prompt sensitivity by temperature range

Why This Matters

 **Definition:** Meaningful extension evidence means output changes remain visible across repeated runs while key controls stay fixed.

 **Lightbulb Takeaway:** Progress comes from asking better questions, not bigger models.

Action 1: What You Learned

- You learned how to scope extension projects with one measurable question and clear controls.
- You learned how to choose extension themes that build directly on your completed lab evidence.
- You learned how to sustain long-term inquiry by documenting limits as well as positive findings.

Action 2: Reflect

- Which extension theme gives you the strongest question for controlled testing?
- What control variables must remain fixed in your chosen extension project?
- What evidence will show that your extension result is meaningful rather than incidental?

Action 3: Do This Next

- Draft a one-page extension proposal with objective, controls, variables, and evidence targets.
- Review proposals with a peer and compare where each plan risks uncontrolled variables.

CHAPTER

Advanced Prompt Engineering Lab

30

In this chapter, you will explore how framing changes outputs, how to build prompt families, and how to compare outputs systematically. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how framing changes outputs
- how to build prompt families
- how to compare outputs systematically

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 30: Advanced Prompt Engineering Lab

About this chapter

This chapter gives you a structured way to test how prompt design affects model behavior under fixed training conditions.

You will run a structured prompt-framing experiment, build prompt families, and compare outputs in a systematic way learners can understand.

What you are going to use

- one checkpoint
- a prompt matrix
- temperature and top-k settings

What you will learn in this chapter

- how framing changes outputs
- how to build prompt families
- how to compare outputs systematically

The work, clearly laid out

1. define three prompt families (narrative, instructional, reflective)
2. generate three outputs per family
3. vary one generation parameter at a time
4. summarize output differences in a table

 **Snippet Purpose:** Narrative framing prompt for baseline language style.

■ **Snippet Type: Bash — Narrative framing prompt for baseline language style**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
"Write a scene where the crew discovers a hidden signal."
--max_new_tokens 60 --temperature 0.8 --top_k 20 --device cpu
```

 **Snippet Purpose:** Instructional framing prompt for stepwise tone.

■ **Snippet Type: Bash — Instructional framing prompt for stepwise tone**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
  "List three steps the crew should follow to investigate a hidden
  signal." --max_new_tokens 60 --temperature 0.8 --top_k 20
--device cpu
```

 **Snippet Purpose:** Reflective framing prompt for reasoning style language.

■ **Snippet Type: Bash — Reflective framing prompt for reasoning style language**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt
  "Reflect on why the crew should be cautious with unknown
  signals." --max_new_tokens 60 --temperature 0.8 --top_k 20
--device cpu
```

Examples of what you might see

■ **Snippet Type: Text Output**

Narrative prompt -> scene description and characters
Instructional prompt -> numbered or imperative style
Reflective prompt -> explanatory and opinionated language

Why This Matters

 **Definition:** Prompt family means a reusable pattern of instruction style.

 **Note:** Prompt effects can be large enough to mask dataset effects if you do not control for them.

 **Lightbulb Takeaway:** Prompt design is part of experimental design.

Action 1: What You Learned

- You learned how prompt family design changes output behavior even when model and data stay fixed.
- You learned how to isolate framing effects by varying one prompt or one generation setting at a time.
- You learned how to report prompt sensitivity with structured comparisons across narrative, instructional, and reflective prompts.

Action 2: Reflect

- Which prompt family produced the largest shift in tone or structure, and why?
- Where did parameter changes matter less than prompt framing in your results?

- What table format made prompt sensitivity easiest to compare across runs?

Action 3: Do This Next

- Build a nine-row prompt matrix (three families by three prompts) and run one controlled comparison pass.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Data Ethics and Attribution

31

In this chapter, you will explore how to document source provenance, how to explain dataset limitations responsibly, and how to teach ethical data choices. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to document source provenance
- how to explain dataset limitations responsibly
- how to teach ethical data choices

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 31: Data Ethics and Attribution

About this chapter

As projects scale, source data quality and attribution become critical.

This chapter also sets a publishing-ready discipline for classroom materials. That includes source provenance, permission status, and a consistent citation style so learners can see responsible technical practice end to end.

In this chapter, you will document source provenance, then explain dataset limitations responsibly, and finally teach ethical data choices. Keep your notes evidence-based so you can explain not only what happened, but why it happened.

What you are going to use

- dataset origin notes
- a source tracking template
- an ethics checklist

What you will learn in this chapter

- how to document source provenance
- how to explain dataset limitations responsibly
- how to teach ethical data choices

The work, clearly laid out

1. record each dataset source
2. note intended learning objective
3. note potential bias or representation gaps
4. include data caveats in presentation

Examples of what you might see

■ Snippet Type: Text Output

Dataset: pirate_dialogue.txt

Source type: fictional script-style writing

Likely bias: exaggerated speech patterns

Teaching note: useful for style shift, not factual QA

Why This Matters

 **Definition:** Provenance means where data came from and how it was prepared.

 **Note:** Good documentation improves trust and reproducibility.

 **Note:** Bias risk examples include one-sided perspective, missing counterexamples, or language that stereotypes groups without context.

 **Note:** Review your current dataset for these bias risk signals before presenting results to learners.

 **Lightbulb Takeaway:** Responsible AI education includes responsible data stories.

Copyright and permissions checklist

- Record whether each source is original, licensed, public-domain, or classroom-fair-use.
- Keep a simple permission note for each non-original text or image.
- Avoid importing unclear or unverified source material into training data.
- Add a one-line rights note when sharing worksheets, slides, or exports.

Citation style for this book and classroom materials

- Use one citation style consistently across your teaching pack (APA 7th is recommended for school settings).
- Include author, year, title, and source URL when citing online references.
- If no author is available, use the organisation name and retrieval date.
- Keep a short references list with every capstone report or presentation.

Action 1: What You Learned

- You learned how provenance notes make dataset choices traceable and easier to justify.
- You learned how to describe bias and representation limits without weakening technical rigor.
- You learned how ethical attribution improves reproducibility, trust, and classroom discussion quality.

Action 2: Reflect

- Which provenance detail is most important when sharing a dataset with learners?

- What bias risk in your current data should be disclosed before interpretation?
- How would missing attribution reduce confidence in your findings?

Action 3: Do This Next

- Create a provenance card for one dataset including source type, objective, and likely bias gaps.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Full Classroom Script Pack

32

In this chapter, you will explore how to explain concepts in beginner language, how to move smoothly between technical steps, and how to maintain engagement. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to explain concepts in beginner language
- how to move smoothly between technical steps
- how to maintain engagement

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 32: Full Classroom Script Pack

About this chapter

This chapter gives complete talk tracks you can use in live teaching.

You will use classroom-ready talk tracks to explain technical steps in plain English, keep transitions smooth, and sustain learner attention.

Treat these scripts as scaffolds, not rigid scripts to perform word-for-word. The goal is consistency of reasoning language so every learner hears the same evidence standards across baseline, retrain, QA, and reflection stages.

What you are going to use

- ready-to-use script blocks
- transition lines between activities
- reflection prompts

What you will learn in this chapter

- how to explain concepts in beginner language
- how to move smoothly between technical steps
- how to maintain engagement

The work, clearly laid out

1. select script set by lesson length
2. rehearse transitions
3. run one script live
4. adjust pacing based on learner response

Examples of what you might see

■ Snippet Type: Text Output

Teacher line: "We are going to keep the model architecture the same and change only data. That way we can observe one clear cause of change."

Why This Matters

 **Definition:** Script pack means pre-written classroom language aligned to workflow steps.

 **Teacher Note:** Scripted transitions are the short bridge lines between phases (prediction -> run -> compare -> debrief) that keep pacing stable.

 **Teacher Note:** Before a live session, mark three "must-say" lines in your chosen script: one line for fairness controls, one for uncertainty framing, and one for claim plus evidence closure.

 **Lightbulb Takeaway:** Clear language is a technical tool.

20-minute quick script

1. "Today we are observing how data changes model behavior."
2. "Watch the baseline output first. Do not judge quality yet; just observe."
3. "Now we retrain with different text and use the same prompt."
4. "What changed? Please cite one exact phrase as evidence."
5. "What stayed similar? That likely comes from architecture and settings."
6. "Final takeaway: data influences style, but reliability still needs review."

35-minute standard script

1. "Before we run, what do you predict will change after retraining?"
2. "We train baseline and capture one output sample."
3. "Now we retrain using a contrasting dataset."
4. "Run same prompt. Compare tone, word choice, and sentence rhythm."
5. "Let's connect this to metrics: loss and perplexity."
6. "Metrics help, but output evidence explains behavior."
7. "We will now test one grounded QA question with context."
8. "Notice how context anchors answers."
9. "Exit reflection: one claim, one piece of evidence."

45-minute extended script

1. "Opening question: What does it mean for a model to learn?"
2. "Baseline training run with narration of each command purpose."

3. "Baseline generation and first evidence note."
4. "Retrain run and second evidence note."
5. "Side-by-side comparison and class discussion."
6. "Mini deep dive: token probabilities in Learn Mode."
7. "Grounded QA demonstration."
8. "Safety framing: fluent text is not automatically true."
9. "Written debrief using claim-evidence format."

Troubleshooting script lines

- "This error is normal and fixable. We will locate the checkpoint path first."
- "If training is slow, we reduce epochs and keep the learning objective."
- "Unexpected output is data for discussion, not failure."

Reflection script lines

- "What changed, and what evidence supports your claim?"
- "What uncertainty remains?"
- "What would you test next to increase confidence?"

Action 1: What You Learned

- You learned how scripted transitions keep lesson flow clear between commands, comparison, and debrief.
- You learned how prewritten lines can translate technical actions into beginner-friendly classroom language.
- You learned how script prompts can keep student discussion focused on claims and evidence.

Action 2: Reflect

- Which script line best moved students from observation to evidence-backed claims?
- Where in your lesson did scripted transitions improve pacing the most?
- Which part of your script needs revision to sound more natural in your own voice?

Action 3: Do This Next

- Adapt one script set to your own classroom style and timing, then rehearse it once out loud.

- Pair with a colleague, run alternating script sections, and compare which lines produced clearer learner responses.

CHAPTER

Student Workbook Section

33

In this chapter, you will explore how to convert model runs into learning artifacts, how to strengthen reasoning through writing, and how to produce assessable student evidence. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to convert model runs into learning artifacts
- how to strengthen reasoning through writing
- how to produce assessable student evidence

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 33: Student Workbook Section

About this chapter

This chapter provides workbook-style pages that can be used directly in class.

By the end of this chapter, you should be able to convert model runs into learning artifacts, strengthen reasoning through writing, and produce assessable student evidence.

What you are going to use

- guided prompts
- comparison tables
- reflection checklists

What you will learn in this chapter

- how to convert model runs into learning artifacts
- how to strengthen reasoning through writing
- how to produce assessable student evidence

The work, clearly laid out

1. complete baseline worksheet
2. complete retrain worksheet
3. complete QA worksheet
4. complete reflection summary

Examples of what you might see

■ Snippet Type: Text Output

Claim: Pirate retrain changed tone.

Evidence: Output includes "matey" and command-style lines.

Confidence: Medium (style signal strong, factuality not tested).

Why This Matters

 **Note:** Written reflection increases retention and improves discussion quality.

 **Lightbulb Takeaway:** Output becomes learning when learners explain it in their own words.

Worksheet A: Baseline Observation

1. Prompt used:
2. First generated sentence:
3. Tone description:
4. Repetition observed (yes/no):
5. One question you still have:

Worksheet B: Retrain Comparison

1. Same prompt reused (yes/no):
2. New vocabulary observed:
3. Tone shift observed:
4. Structure shift observed:
5. Most convincing evidence line:

Worksheet C: Metric Reflection

1. Baseline loss:
2. Retrain loss:
3. Baseline perplexity:
4. Retrain perplexity:
5. What do metrics suggest?
6. What do metrics not prove?

Worksheet D: QA Grounding

1. Question asked:
2. Context used:
3. Answer returned:
4. Groundedness score (1-5):
5. If weak, what context change would you make?

Worksheet E: Claim-Evidence Summary

1. Claim 1:

2. Evidence for claim 1:
3. Claim 2:
4. Evidence for claim 2:
5. Uncertainty statement:
6. Next experiment step:

Action 1: What You Learned

- You learned how each workbook sheet turns model output into evidence you can assess, not just observe.
- You learned how the claim-evidence and uncertainty prompts push you to justify conclusions from baseline, retrain, and QA runs.
- You learned how groundedness scoring and metric reflection help you separate style changes from reliability claims.

Action 2: Reflect

- Which worksheet gave you the strongest evidence for a real behavior shift, and what made that evidence strong?
- Where did your confidence score and your written uncertainty statement disagree, and why?
- In Worksheet D, what specific context edit would most improve your groundedness score?

Action 3: Do This Next

- Run one new prompt through Worksheets A, B, and E, then write a tighter claim using only one sentence of evidence.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Capstone Project Handbook

34

In this chapter, you will explore how to scope a capstone well, how to present findings professionally, and how to communicate limits honestly. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to scope a capstone well
- how to present findings professionally
- how to communicate limits honestly

Applied Practice: Run guided labs and evidence-rich tasks.

Chapter 34: Capstone Project Handbook

About this chapter

This chapter helps learners build and present a complete final project.

You will design a credible capstone scope, present results with professional structure, and communicate limits honestly alongside strengths.

The capstone should show method maturity, not just interesting outputs. A strong project makes controls explicit, explains evidence quality, and states limits in the same level of detail as findings.

What you are going to use

- capstone planning template
- run logs
- output comparison portfolio

What you will learn in this chapter

- how to scope a capstone well
- how to present findings professionally
- how to communicate limits honestly

The work, clearly laid out

1. define project question
2. design controlled experiment
3. run baseline and variant workflows
4. collect outputs and metrics
5. present claim-evidence conclusion

Examples of what you might see

■ Snippet Type: Text Output

Question: How does poetic training data affect response rhythm?
 Method: fixed architecture + fixed prompt set + dataset swap
 Finding: higher metaphor density and shorter clause structure
 Limit: factual reliability not improved by style-focused data

Why This Matters

 **Definition:** Capstone is a culminating project demonstrating method mastery.

 **Definition:** Evidence chain means linking commands, outputs, metrics, and one limitation statement into a single justified claim.

 **Note:** Capstone scoring improves when learners rehearse a two-minute method summary before final delivery. This makes controls and variable choices easier for reviewers to follow.

 **Lightbulb Takeaway:** A strong capstone explains both what changed and what remains uncertain.

Capstone template (one-page)

1. Project title:
2. Research question:
3. Datasets used:
4. Fixed controls:
5. Variables changed:
6. Commands run:
7. Key outputs:
8. Metrics summary:
9. Main claim:
10. Supporting evidence:
11. Limitation statement:
12. Recommended next test:

Presentation rubric (10-point)

1. Clear question (2 pts)
2. Controlled method (2 pts)
3. Evidence quality (2 pts)

4. Honest limitations (2 pts)
5. Delivery clarity (2 pts)

Action 1: What You Learned

- You learned how to frame a capstone question that is specific enough to test with fixed controls.
- You learned how to connect commands, outputs, and metrics into one coherent claim-evidence story.
- You learned how to report limits honestly so your project conclusions stay credible.

Action 2: Reflect

- Which fixed control in your capstone design protected your conclusion the most?
- Which part of your evidence chain felt weakest: outputs, metrics, or limitation statement?
- What is one claim from your capstone that you can support with two different kinds of evidence?

Action 3: Do This Next

- Draft a second capstone question that changes only one variable from your first plan.
- Present your one-page capstone template to a peer and compare which rubric category each of you scored highest.

CHAPTER

Implementation Workbook (Week-by-Week)

35

In this chapter, you will explore how to spread learning across multiple sessions, how to track progress week by week, and how to integrate practice and reflection. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to spread learning across multiple sessions
- how to track progress week by week
- how to integrate practice and reflection

Operations: Keep lessons reliable, measurable, and calm.

Chapter 35: Implementation Workbook (Week-by-Week)

About this chapter

This chapter turns the full book into a practical multi-week teaching schedule.

You will convert the book into a realistic multi-week plan, track progress with weekly artifacts, and balance practice with reflection across the term.

The aim is not to "fit everything in." The aim is to pace learning so each week finishes with one visible result learners can explain in plain language.

What you are going to use

- weekly lesson map
- checkpoint milestones
- reflection checkpoints

What you will learn in this chapter

- how to spread learning across multiple sessions
- how to track progress week by week
- how to integrate practice and reflection

The work, clearly laid out

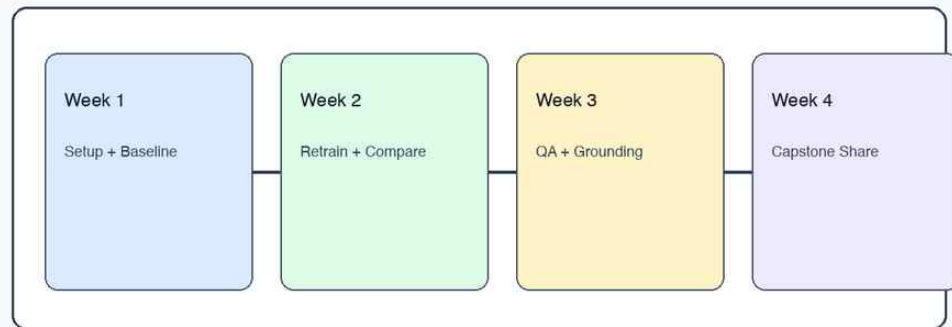
1. choose 4-week or 6-week format
2. map chapters to weekly goals
3. define weekly evidence artifacts
4. plan mid-point and final review

Examples of what you might see

■ Snippet Type: Text Output

```
Week 1: setup + baseline run
Week 2: retrain + comparison
Week 3: QA mode + grounding
Week 4: capstone presentation
```

Week-by-Week Delivery Map



Each week should end with one clear artifact and one clear explanation.

Alt text: Weekly implementation map showing four linked weeks: setup, retrain, QA grounding, and capstone presentation.

Figure 6. Weekly pacing works best when each week ends with one artifact and one explanation.

Why This Matters

Definition: Cognitive load is the amount of mental effort needed in one session. If a week combines too many new tasks, split delivery and keep one core goal.

Note: Use weekly deliverables as checkpoints: if a deliverable is weak, move the next milestone later and add one reinforcement activity first.

Lightbulb Takeaway: Long-term learning works best when each week produces a small, concrete artifact.

Four-week implementation map

Week 1 objectives

- install and verify environment
- run first baseline model
- capture first output evidence

Week 1 deliverables

- successful environment check
- one baseline checkpoint
- one prompt/output reflection sheet

Week 2 objectives

- run retrain contrast workflow
- compare fixed prompt outputs
- discuss style and reliability differences

Week 2 deliverables

- retrain checkpoint
- side-by-side output comparison notes
- claim-evidence summary

Week 3 objectives

- build QA corpus and train QA checkpoint
- run grounded QA questions
- inspect Learn Mode visuals

Week 3 deliverables

- QA run log
- groundedness reflection
- one Learn Mode observation note

Week 4 objectives

- design mini-project
- run final comparison
- present findings and limitations

Week 4 deliverables

- capstone one-pager
- presentation slides or poster
- reflection on next experiment

Six-week implementation map**Week 1**

- onboarding, setup, baseline run

Week 2

- retrain workflow and fair comparisons

Week 3

- deeper evaluation and metric interpretation

Week 4

- QA workflow and context engineering

Week 5

- guided lab extensions and troubleshooting

Week 6

- capstone delivery and peer review

Action 1: What You Learned

- You learned how to map chapters into weekly objectives that produce visible progress artifacts.
- You learned how the four-week and six-week plans change pacing while keeping the same core learning outcomes.
- You learned how to use deliverables to catch learning gaps early instead of waiting for the final presentation.

Action 2: Reflect

- Which week in your chosen plan carries the highest cognitive load, and how will you reduce it?
- Which deliverable gives the clearest signal that students understood grounded QA?
- What is one milestone you would move earlier or later after reviewing your class timing?

Action 3: Do This Next

- Build a custom five-week schedule using the existing objectives and write one success signal for each week.
- Compare your schedule with another teacher or learner and agree on one shared checkpoint both plans should include.

CHAPTER

Frequently Asked Questions (Classroom Edition)

36

In this chapter, you will explore how to answer common conceptual questions, how to handle confusion points quickly, and how to keep explanations grounded and consistent. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to answer common conceptual questions
- how to handle confusion points quickly
- how to keep explanations grounded and consistent

Operations: Keep lessons reliable, measurable, and calm.

Chapter 36: Frequently Asked Questions (Classroom Edition)

About this chapter

This chapter provides practical answers to the questions teachers and students ask most often.

You will use these FAQs to handle recurring confusion quickly while keeping explanations consistent across sessions. Each answer is designed to connect a common claim to observable evidence and safer interpretation language.

What you are going to use

- FAQ prompts
- short answer explanations
- decision guides

What you will learn in this chapter

- how to answer common conceptual questions
- how to handle confusion points quickly
- how to keep explanations grounded and consistent

The work, clearly laid out

1. review FAQs before teaching
2. pick answers that match learner level
3. use examples from your own runs

Examples of what you might see

■ Snippet Type: Text Output

Q: If loss goes down, does that mean the model is "smart"?

A: It means better fit to that dataset, not human understanding.

Why This Matters

📖 **Note:** FAQ pairing works best when one answer addresses metrics (for example loss or perplexity) and the second addresses evidence language in output discussion.

📖 **Note:** A practical pairing example is Q4 (low perplexity still wrong) with Q7 (model sounds fluent but does not "understand").

👤 **Teacher Note:** In a live debrief, pairing one metrics FAQ with one evidence-language FAQ usually produces clearer discussion than using either in isolation.

📖 **Note:** If a question is not covered here, answer in the same pattern: short definition, one evidence anchor, one limit statement, and one suggested next check.

💡 **Lightbulb Takeaway:** Good answers are short, honest, and evidence-linked.

FAQ set

Q1: Why does the model repeat itself?

A: Tiny models have limited capacity and may lock into short loops, especially when training data is narrow or prompts are open-ended.

Q2: Why do we keep the prompt fixed?

A: So that output differences are more likely caused by data changes instead of prompt wording changes.

Q3: Why is context so important in QA mode?

A: Context provides grounding. Without it, the model may produce plausible but unsupported answers.

Q4: Can a low perplexity model still be wrong?

A: Yes. Perplexity indicates predictability on a dataset, not factual accuracy across all topics.

Q5: Why does a pirate-trained model affect even neutral prompts?

A: Training shifts token likelihood patterns, so style cues appear even when the prompt is neutral.

Q6: Should we train for more epochs by default?

A: Not always. For classroom goals, shorter runs are often enough to show key behavior changes without overfitting or long delays.

Q7: What if students think the model "understands"?

A: Ask them to provide evidence and distinguish fluent pattern generation from human-like understanding.

Q8: What if outputs look random?

A: Check data quality, context quality, and generation settings. Then rerun a small controlled experiment.

Q9: How do I keep lessons inclusive for mixed ability groups?

A: Use role-based tasks: some students run commands, others annotate outputs, others lead evidence review.

Q10: What is a quick success criterion for a lesson?

A: Students can state one claim and one supporting output line, plus one limitation statement.

Action 1: What You Learned

- You learned how to answer recurring AI questions with short explanations tied to observed classroom behavior.
- You learned how to correct common misunderstandings, such as equating low loss with true understanding, without overcomplicating the explanation.
- You learned how to use FAQ responses to keep discussions evidence-based when students make broad claims.

Action 2: Reflect

- Which FAQ answer would most help if a student says, "The model is smart because it sounds fluent"?
- Which question in this chapter needs a local classroom example from your own runs to land better?
- Which two FAQ entries are most useful to pair during a live debrief, and why?

Action 3: Do This Next

- Write one new FAQ and answer based on a confusion point from your last session.

- In pairs, role-play a student question and teacher response, then compare which wording felt clearer and more accurate.

CHAPTER

Teacher Quick-Reference Cards

37

In this chapter, you will explore how to respond quickly during lessons, how to keep language concise under pressure, and how to maintain flow during demos. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to respond quickly during lessons
- how to keep language concise under pressure
- how to maintain flow during demos

Operations: Keep lessons reliable, measurable, and calm.

Chapter 37: Teacher Quick-Reference Cards

About this chapter

This chapter gives compact reference cards for live teaching moments.

You will build quick-reference cards that help you respond fast in live lessons, keep language concise under pressure, and preserve lesson flow.

These cards are operational tools. They reduce cognitive load during delivery so you can spend attention on learner reasoning, not command recall.

What you are going to use

- quick command cards
- quick explanation cards
- quick troubleshooting cards

What you will learn in this chapter

- how to respond quickly during lessons
- how to keep language concise under pressure
- how to maintain flow during demos

The work, clearly laid out

1. print or copy quick-reference cards
2. keep one card set visible during demos
3. use cards for rapid transitions and recovery

Examples of what you might see

■ Snippet Type: Text Output

Card: "Baseline Run"

Goal: produce first checkpoint and one output sample.

Why This Matters

 **Definition:** A quick-reference card is a pre-validated command or explanation unit that can be used without rewriting language under time pressure.

 **Teacher Note:** Keep cards physically grouped by lesson phase (setup, run, compare, debrief). Phase grouping lowers decision time when a session becomes busy.

 **Lightbulb Takeaway:** Fast reference tools reduce teaching cognitive load.

Card set A: Command cards

Card A1: Baseline training

 **Snippet Purpose:** Quick card for creating a baseline checkpoint during live teaching.

 **Snippet Type:** Bash — Quick card for creating a baseline checkpoint during live teaching

```
kairo-train --input_file data/samples/space_adventure.txt
--out_dir runs/card_baseline --epochs 1 --batch_size 4 --seq_len
32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

Card A2: Baseline generation

 **Snippet Purpose:** Quick card for generating a baseline output sample on demand.

 **Snippet Type:** Bash — Quick card for generating a baseline output sample on demand

```
kairo-generate --checkpoint runs/card_baseline/best.pt --prompt
"The station doors opened slowly" --max_new_tokens 40
--temperature 0.8 --top_k 20 --device cpu
```

Card A3: Retrain contrast

 **Snippet Purpose:** Quick card for training the contrast model with pirate-style data.

 **Snippet Type:** Bash — Quick card for training the contrast model with pirate-style data

```
kairo-train --input_file data/samples/pirate_dialogue.txt
--out_dir runs/card_pirate --epochs 1 --batch_size 4 --seq_len
32 --d_model 64 --n_heads 4 --n_layers 2 --device cpu
```

Card A4: QA quick run

 **Snippet Purpose:** Quick card for demonstrating context-grounded QA in class.

■ Snippet Type: Bash — Quick card for demonstrating context-grounded QA in class

```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "Who  
pilots the Aurora?" --context "Captain Rowan is the pilot of the  
starship Aurora."
```

Card set B: Explanation cards

- "Same architecture, different data, different behavior."
- "Fluent output is not guaranteed truth."
- "Context improves grounded QA answers."
- "Metrics guide us, but output evidence explains behavior."

Card set C: Troubleshooting cards

- Missing checkpoint: verify run directory and best . pt path.
- Slow training: reduce epochs or sequence length.
- Noisy QA output: add clearer context.
- Unexpected style: check which dataset trained the checkpoint.

Card set D: Reflection cards

- "What changed?"
- "What evidence proves it?"
- "What remains uncertain?"
- "What would you test next?"

 **Note:** The most specific reflection prompts ask for one exact output line plus one limitation statement.

 **Note:** Use the reflection card prompts during debrief so evidence statements stay specific and comparable across groups.

 **Note:** Reflection card prompts are designed to produce specific student evidence statements, not general opinions.

Action 1: What You Learned

- You learned how command cards reduce demo friction by keeping key runs immediately available.
- You learned how explanation and troubleshooting cards help you recover quickly when outputs or checkpoints do not match expectations.
- You learned how reflection cards keep student discussion focused on claims, evidence, and next tests during live sessions.

Action 2: Reflect

- Which card from Set A would you place first in a lesson, and what transition line would you say before using it?
- Which troubleshooting card would most likely save time in your current teaching setup?
- Which reflection card prompt produces the most specific student evidence statements in your group?

Action 3: Do This Next

- Build a one-page quick-reference sheet by selecting only six cards you will actually use next session.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Deployment and Operations Checklist

38

In this chapter, you will explore how to prepare a stable classroom deployment, how to avoid common operational pitfalls, and how to recover quickly when issues appear. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to prepare a stable classroom deployment
- how to avoid common operational pitfalls
- how to recover quickly when issues appear

Operations: Keep lessons reliable, measurable, and calm.

Chapter 38: Deployment and Operations Checklist

About this chapter

This chapter helps you move from workshop experiments to repeatable operational delivery in schools or clubs.

You will prepare a stable classroom deployment with preflight checks, fallback steps, and clear recovery routines for common runtime issues.

What you are going to use

- deployment checklist
- run directory conventions
- backup and recovery plan

What you will learn in this chapter

- how to prepare a stable classroom deployment
- how to avoid common operational pitfalls
- how to recover quickly when issues appear

The work, clearly laid out

1. standardize project folder structure
2. prepare known-good checkpoints
3. prepare backup command cards
4. preflight the environment before sessions
5. capture run logs after each class

Examples of what you might see

■ Snippet Type: Text Output

Deployment status:

- environment verified
- baseline checkpoint present
- QA checkpoint present
- fallback script prepared

Why This Matters

 **Definition:** Operational readiness means the lesson can run reliably for learners with predictable outcomes and recoverable failure modes.

 **Note:** Reliability is not just model quality. It is also process quality.

 **Note:** Postflight evidence should include saved outputs, metrics, and one improvement note that will change the next session plan.

 **Lightbulb Takeaway:** A good deployment plan saves your teaching time for learning, not debugging.

Session preflight checklist

1. virtual environment activates successfully
2. `kairo-train --help` and `kairo-generate --help` run
3. baseline and retrain checkpoints exist
4. one prompt test runs in under expected time
5. presentation prompts and reflection questions are ready

Session postflight checklist

1. save outputs in dated folder
2. archive metrics and notes
3. record one lesson improvement for next session
4. verify cleanup and restore known-good state

Command health checks

 **Snippet Purpose:** Verify CLI commands are available before class starts.

■ **Snippet Type:** Bash — Verify CLI commands are available before class starts

```
kairo-train --help
kairo-generate --help
kairo-evaluate --help
kairo-qa --help
```

 **Snippet Purpose:** Verify a known-good baseline checkpoint is loadable.

■ **Snippet Type:** Bash — Verify a known-good baseline checkpoint is loadable

```
kairo-generate --checkpoint runs/card_baseline/best.pt --prompt  
"System check prompt" --max_new_tokens 12 --device cpu
```

Failure mode playbook

- **Missing checkpoint:** use known-good fallback checkpoint card.
- **Slow runtime:** reduce `--max_new_tokens` and skip long retrains.
- **Environment mismatch:** re-activate `.venv` and rerun help checks.
- **Noisy QA answer:** switch to context-first QA examples.

Action 1: What You Learned

- You learned how preflight checks catch missing tools and checkpoints before learners are waiting.
- You learned how postflight routines preserve outputs and notes so each class improves the next one.
- You learned how a failure-mode playbook turns common runtime issues into planned recovery steps.

Action 2: Reflect

- Which preflight check in this chapter would most likely prevent a full lesson delay?
- In your environment, which failure mode is highest risk and what is your first fallback action?
- Which postflight item gives you the most useful evidence for improving the next session?

Action 3: Do This Next

- Run a full dry-run using the checklist and record the first point where timing slips.
- Pair with a colleague to simulate one failure mode and compare your recovery paths for speed and clarity.

CHAPTER

Accessibility and Inclusive Teaching Patterns

39

In this chapter, you will explore how to support mixed experience levels, how to design equitable participation, and how to reduce cognitive overload. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to support mixed experience levels
- how to design equitable participation
- how to reduce cognitive overload

Operations: Keep lessons reliable, measurable, and calm.

Chapter 39: Accessibility and Inclusive Teaching Patterns

About this chapter

This chapter helps you make Kairo lessons inclusive for diverse learners.

You will design lessons that work for mixed-experience groups by assigning clear roles, reducing cognitive overload, and widening participation options.

Inclusion here is practical and measurable. More learners should be able to contribute evidence, explain a result, and complete reflection tasks without being blocked by one mode of participation.

What you are going to use

- differentiated task roles
- accessibility checkpoints
- alternative output formats

What you will learn in this chapter

- how to support mixed experience levels
- how to design equitable participation
- how to reduce cognitive overload

The work, clearly laid out

1. assign multi-role team structure
2. provide visual and text alternatives
3. scaffold reflection prompts by level
4. allow multiple evidence formats
5. debrief inclusively

Examples of what you might see

■ Snippet Type: Text Output

Role split:

- operator: runs commands
- observer: records output patterns
- analyst: writes claim-evidence notes
- presenter: shares findings

Why This Matters

 **Definition:** Inclusive design means activities are accessible across prior skill, language confidence, and learning preference.

 **Note:** A learner can contribute meaningfully without typing every command.

 **Note:** If an accessibility checklist item needs planning, set a simple date and owner so it becomes part of your next delivery plan.

 **Note:** Match assessment format to learner needs: written for detail, verbal for confidence building, visual for comparison-heavy thinking.

 **Note:** Build accessibility into planning, not rescue. Set role options, format options, and sentence scaffolds before class starts so support is predictable.

 **Lightbulb Takeaway:** Inclusion improves technical quality because more perspectives examine the evidence.

Accessibility checklist

1. explain each command purpose aloud
2. provide printed workflow cards
3. display outputs with readable contrast and zoom
4. avoid jargon without definition
5. offer sentence starters for reflections

Reflection scaffolds

- beginner: "One thing that changed was ____."
- intermediate: "The output changed because ____ evidence shows ____."
- advanced: "A limitation of this result is ____, so I would test ____ next."

Inclusive assessment options

- written response

- verbal explanation
- visual comparison board
- paired explanation interview

Action 1: What You Learned

- You learned how role-based participation lets learners contribute meaningfully even when coding confidence differs.
- You learned how accessibility supports, like clear contrast and sentence starters, reduce cognitive overload during technical tasks.
- You learned how inclusive assessment formats capture understanding in more than one communication style.

Action 2: Reflect

- Which role assignment pattern in this chapter would best support your current group mix?
- Which accessibility checklist item is easiest to implement immediately, and which needs planning?
- Which assessment format would reveal understanding for a learner who struggles with written reflection?

Action 3: Do This Next

- Redesign one existing activity with explicit operator, observer, analyst, and presenter roles.
- Run the redesigned activity with a partner group and compare whether role clarity improved contribution balance.

CHAPTER

Continuous Improvement Plan

40

In this chapter, you will explore how to improve lesson quality each cycle, how to prioritize high-value changes, and how to track impact over time. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to improve lesson quality each cycle
- how to prioritize high-value changes
- how to track impact over time

Operations: Keep lessons reliable, measurable, and calm.

Chapter 40: Continuous Improvement Plan

About this chapter

This chapter helps you evolve the curriculum over multiple cohorts.

You will improve lesson quality each cycle, prioritize high-value changes, and track impact over time with clear evidence. The goal is disciplined iteration: small changes, explicit metrics, and honest review of what did or did not work.

You are building a steady improvement habit here. One measured change per cycle is more reliable than many untracked adjustments made all at once.

What you are going to use

- retrospective notes
- quality metrics
- student feedback loops

What you will learn in this chapter

- how to improve lesson quality each cycle
- how to prioritize high-value changes
- how to track impact over time

The work, clearly laid out

1. collect post-session evidence
2. identify bottlenecks and confusion points
3. prioritize one improvement per cycle
4. test improvement in next delivery
5. compare outcomes and iterate

Examples of what you might see

■ Snippet Type: Text Output

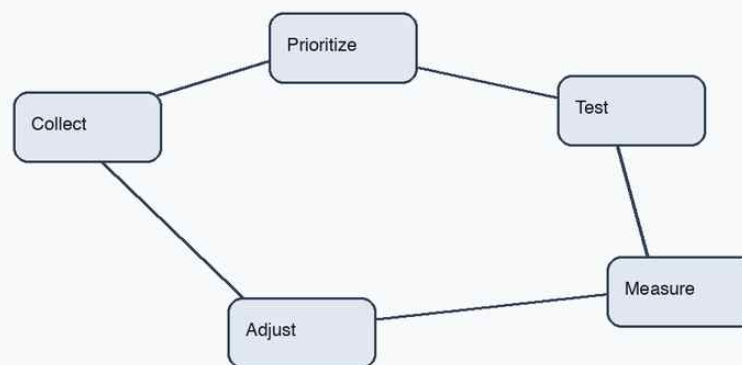
Cycle note:

Issue: students confused by prompt fairness.

Change: added fixed-prompt worksheet.

Result: stronger evidence quality in reflections.

Continuous Improvement Cycle



Improve one thing at a time, then prove whether it helped.

Alt text: Continuous improvement cycle showing collect, prioritize, test, measure, and adjust as a repeating loop.

Figure 7. Improvement becomes reliable when each cycle closes with a measured result.

Why This Matters

Definition: Retrospective means structured review of what worked, what failed, and what to improve next.

Note: Small iterative improvements compound quickly over a term.

Note: Decide your reset rule in advance. For example: if two consecutive cycles show no movement in evidence quality and confidence, redesign the workflow rather than applying another small patch.

Lightbulb Takeaway: Treat teaching quality like model quality—measure, adjust, repeat.

Improvement dashboard template

1. Session date:
2. Objective met (yes/no):
3. Evidence quality score (1-5):
4. Student confidence score (1-5):

5. Biggest friction point:
6. Improvement chosen:
7. Next session success signal:

Change prioritization rubric

- high impact, low effort: do immediately
- high impact, medium effort: schedule next cycle
- low impact, high effort: defer unless required

Action 1: What You Learned

- You learned how to run a repeatable improvement cycle by linking observed friction to one concrete change.
- You learned how the prioritization rubric helps you choose high-impact adjustments instead of reacting to every issue at once.
- You learned how dashboard signals, such as evidence quality and confidence, show whether your changes actually worked.

Action 2: Reflect

- Which session metric showed the clearest improvement since you started teaching with Kairo?
- What one change to your delivery would most reduce repeat troubleshooting in future sessions?
- How will you decide when your improvement plan needs a full reset rather than a small adjustment?

Action 3: Do This Next

- Complete one improvement dashboard entry using data from your most recent session.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

CHAPTER

Glossary (Terms and Parameters)

41

In this chapter, you will explore what each key concept means, what each command parameter controls, and how to explain workflow language clearly. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- what each key concept means
- what each command parameter controls
- how to explain workflow language clearly

Reference: Use this section as a quick teaching lookup.

Chapter 41: Glossary (Terms and Parameters)

About this chapter

This chapter is a complete meaning guide for technical terms and command parameters used across the whole book.

You will use this glossary to secure your vocabulary, connect terms to their chapter examples, and decode command parameters before reruns.

Treat this chapter as your interpretation safety net. When a result feels confusing, come here first, confirm term meaning, then return to the chapter example with clearer language.

What you are going to use

- term definitions
- parameter definitions
- quick lookup while practicing

What you will learn in this chapter

- what each key concept means
- what each command parameter controls
- how to explain workflow language clearly

The work, clearly laid out

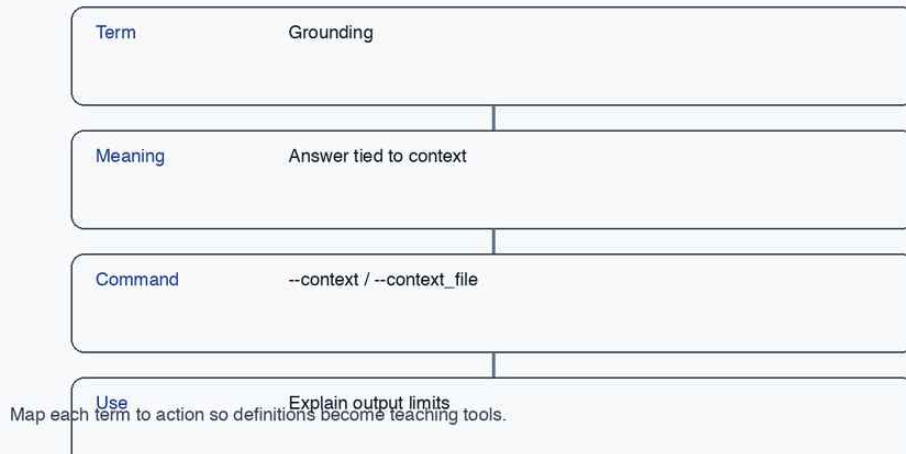
1. review unknown terms
2. map terms to chapters where used
3. review parameter meanings before reruns

Examples of what you might see

■ Snippet Type: Text Output

```
--epochs 1: one pass through training data  
--device cpu: use CPU instead of GPU
```

Glossary to Classroom Use



Alt text: Glossary lookup map showing how a term moves from definition to command usage and classroom explanation.

Figure 8. The glossary is most useful when each term is tied to a command and a teaching use-case.

Why This Matters

💡 Lightbulb Takeaway: Shared vocabulary makes collaboration and teaching easier.

📖 Note: A useful glossary reflection compares how you used a term before this chapter versus how you use it now with a concrete example.

Core terms

- **Attention:** Mechanism that weighs prior tokens when predicting the next token (see Chapter 8).
- **Batch:** Group of sequences processed in one training step.
- **Checkpoint:** Saved model state such as best .pt (see Chapters 5 and 13).
- **Context:** Reference text supplied before generation (see Chapters 7 and 25).
- **Corpus:** Prepared text collection used for training.
- **Dataset:** Source text used for training or evaluation (see Chapters 5 and 15).
- **Evidence:** Specific output, metric, or observation used to support a claim (see Chapters 1 and 6).
- **Epoch:** One full pass through the training data.
- **Grounding:** Anchoring output to supplied context (see Chapters 7 and 25).
- **Inference:** Running a trained model to generate output.
- **Loss:** Numerical error signal used in training/evaluation (see Chapters 6 and 18).

- **Model architecture:** Structural design (layers, heads, model width).
- **Next-token prediction:** Predicting the next token from prior tokens.
- **Perplexity:** Uncertainty measure derived from loss (see Chapters 6 and 19).
- **Prompt:** Input text to start generation.
- **Provenance:** Record of where source data came from and how it was prepared.
- **Rubric:** Structured scoring guide for evaluating learner work.
- **Capstone:** Final synthesis project that demonstrates end-to-end mastery.
- **Retraining:** Training again on new or changed data (see Chapters 5 and 24).
- **Token:** Unit of text processed by the model (byte-based in Kairo; see Chapter 17).
- **Validation:** Performance check on held-out data.

Parameter glossary (entire book)

- `--input_file`: Path to training or evaluation text file.
- `--out_dir`: Directory for outputs such as checkpoints and logs.
- `--epochs`: Number of full training passes through dataset.
- `--batch_size`: Number of training sequences per optimization step.
- `--seq_len`: Token window length used by training/generation pipeline.
- `--d_model`: Model hidden dimension (representation width).
- `--n_heads`: Number of attention heads per transformer block.
- `--n_layers`: Number of transformer layers.
- `--device`: Hardware target, commonly `cpu` or `cuda`.
- `--checkpoint`: Path to saved model checkpoint for loading.
- `--prompt`: Input seed text for generation.
- `--max_new_tokens`: Maximum number of generated tokens to append.
- `--temperature`: Sampling randomness control (higher is more random).
- `--top_k`: Restrict sampling to top-k probable next tokens.
- `--input_jsonl`: JSONL source for QA corpus conversion.
- `--output_file`: File path for generated corpus output.
- `--question`: User question string for QA mode.
- `--context`: Inline context text used to ground QA response.
- `--context_file`: Path to context text file for QA grounding.
- `--help`: Shows command usage and available options without running the full action.

Action 1: What You Learned

- You learned how to decode core AI terms in this book so chapter instructions are easier to follow.
- You learned how parameter meanings, such as --epochs and --temperature, map directly to training and generation behavior.
- You learned how a shared glossary supports clearer explanations when teaching or collaborating.

Action 2: Reflect

- Which three glossary terms do you now use differently because their definitions are clearer?
- Which parameter from this chapter most changed how you interpret a training or generation result?
- Where in the book would adding a glossary callout help beginners connect concept to command faster?

Action 3: Do This Next

- Create a personal mini-glossary of ten terms you want to master and write one example for each.
- Discuss with your group: what would change if you altered one variable in this step?

CHAPTER

Conclusion

42

In this chapter, you will explore what core skills you now have, how to carry those skills into future projects, and how to continue learning without overwhelm. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- what core skills you now have
- how to carry those skills into future projects
- how to continue learning without overwhelm

Reference: Use this section as a quick teaching lookup.

Chapter 42: Conclusion

About this chapter

You have now completed a full beginner-to-practice AI journey. This conclusion helps you anchor what matters most so your confidence lasts beyond this book.

By the end of this chapter, you should be able to identify the core skills you now have, carry those skills into future projects, and continue learning without overwhelm.

What you are going to use

- your completed chapter notes
- your saved command outputs
- your reflections from guided labs

What you will learn in this chapter

- what core skills you now have
- how to carry those skills into future projects
- how to continue learning without overwhelm

The work, clearly laid out

1. review your strongest evidence moments
2. identify one skill you can now explain clearly
3. choose one next project to continue your growth

Examples of what you might see

■ Snippet Type: Text Output

```
"I can now compare baseline and retrained output using  
evidence."
```

```
"I can explain why scope, data, and prompt design change model  
behavior."
```

Why This Matters

💡 **Lightbulb Takeaway:** Real confidence is the ability to explain why something changed, not just to say that it changed.

You have practiced observation, comparison, and explanation repeatedly. That is the core of technical confidence. These habits will transfer to new tools, larger models, and more advanced AI systems.

 **Note: Workflow stability means you can repeat baseline, retrain, evaluate, and grounded QA with clear notes and minimal troubleshooting.**

Action 1: What You Learned

- You learned how to summarize your growth by pointing to concrete outputs, comparisons, and reflections from across the book.
- You learned how baseline training, retraining, evaluation, and QA grounding connect into one complete workflow.
- You learned how to choose a realistic next project so your confidence continues through practice.

Action 2: Reflect

- Which chapter produced the strongest evidence that your reasoning skills improved, and what proves it?
- Which part of the end-to-end workflow still feels least stable for you in real use?
- What is your first follow-on project, and which two chapters will you revisit before starting it?

Action 3: Do This Next

- Write a one-page personal summary that links one skill, one evidence example, and one next-step project.
- Write a one-paragraph summary of this step in your own words, without referring to the chapter.

References (APA 7th Edition)

- Hmelo-Silver, C. E. (2004). Problem-based learning: What and how do students learn? *Educational Psychology Review*, 16(3), 235-266.
<https://doi.org/10.1023/B:EDPR.0000034022.16470.f3>
- Kolb, D. A. (1984). *Experiential learning: Experience as the source of learning and development*. Prentice Hall.

Further Reading

- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 610-623.
<https://doi.org/10.1145/3442188.3445922>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877-1901.
<https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- OpenAI. (2023). GPT-4 technical report. arXiv. <https://arxiv.org/abs/2303.08774>
- Touretzky, D. S., Gardner-McCune, C., Martin, F., & Seehorn, D. (2019). Envisioning AI for K-12: What should every child know about AI? Proceedings of the AAAI Conference on Artificial Intelligence, 33(1), 9795-9799.
<https://doi.org/10.1609/aaai.v33i01.33019795>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. arXiv.
<https://arxiv.org/abs/1706.03762>

Web Resources

- AI4K12 Initiative. (n.d.). Five big ideas in AI. <https://ai4k12.org>
- MIT RAISE. (n.d.). Day of AI curriculum resources. <https://raise.mit.edu/dayofai>
- OpenAI. (n.d.). OpenAI research index. <https://openai.com/research>
- UNESCO. (2021). AI and education: Guidance for policy-makers.
<https://unesdoc.unesco.org>

CHAPTER

Key Words Index

43

In this chapter, you will explore how to locate core concepts quickly, how to revise efficiently before teaching or presenting, and how to connect terms back to practical examples. By the end, you should be able to explain how these ideas work together in practice.

What you will walk away with

- how to locate core concepts quickly
- how to revise efficiently before teaching or presenting
- how to connect terms back to practical examples

Reference: Use this section as a quick teaching lookup.

Chapter 43: Key Words Index

About this chapter

This chapter gives you a fast lookup list of important terms and where they appear in the book.

Use this chapter to build a repeatable habit: locate core concepts quickly, revise efficiently before teaching or presenting, and connect terms back to practical examples. That sequence will help you connect hands-on steps to clear reasoning.

Use it like a route map, not a final destination. The goal is to find the right chapter fast, then revisit the full explanation and evidence in context.

What you are going to use

- alphabetical term list
- page references
- glossary links for full definitions

What you will learn in this chapter

- how to locate core concepts quickly
- how to revise efficiently before teaching or presenting
- how to connect terms back to practical examples

The work, clearly laid out

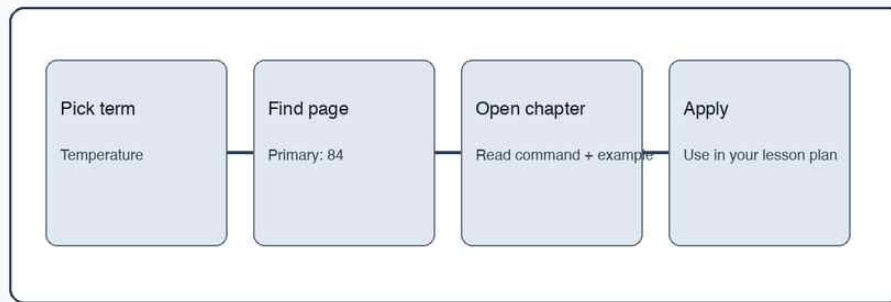
1. choose a term you want to review
2. jump to the listed page(s)
3. revisit the chapter example and explanation

Examples of what you might see

■ Snippet Type: Text Output

Use the generated index below for literal page numbers from this exact build.

Index Lookup Workflow



Use the index first, then jump to the chapter where the term is taught in context.

Alt text: Index lookup workflow showing term selection, page lookup, chapter revisit, and practical application in lesson planning.

Figure 9. A strong index workflow starts with the term, then returns to the chapter where it is taught.

Why This Matters

Note: Page references are generated from the final printable layout.

Note: A cross-reference is strongest when it points to the exact chapter example that first explains the term in practice.

Lightbulb Takeaway: A strong index turns a good book into a useful working tool.

Key words and page numbers

Note: Use this index to jump quickly to where each term is taught.

Format: primary pages show the best starting points; full references show complete page ranges.

- Attention: Primary: 40 | Full: 40-171
- Batch: Primary: 27 | Full: 27-165
- Batch size (--batch_size): Primary: 30 | Full: 30-166
- Capstone: Primary: 136 | Full: 136-166
- Checkpoint: Primary: 30 | Full: 30-166
- Context: Primary: 38 | Full: 38-173
- Corpus: Primary: 37 | Full: 37-166
- Dataset: Primary: 19 | Full: 19-166
- Epoch: Primary: 73 | Full: 73-165
- Epoch (--epochs): Primary: 30 | Full: 30-167

- Evidence: Primary: 16 | Full: 16-173
- Grounding: Primary: 38 | Full: 38-170
- Inference: Primary: 37 | Full: 37-165
- Loss: Primary: 31 | Full: 31-166
- Model architecture: Primary: 31 | Full: 31-166
- Next-token prediction: Primary: 42 | Full: 42-166
- Overfitting: Primary: 73 | Full: 73-144
- Perplexity: Primary: 31 | Full: 31-166
- Prompt: Primary: 21 | Full: 21-169
- Prompt (--prompt): Primary: 21 | Full: 21-169
- Provenance: Primary: 119 | Full: 119-166
- Reliability: Primary: 84 | Full: 84-153
- Retraining: Primary: 19 | Full: 19-170
- Rubric: Primary: 35 | Full: 35-166
- Style transfer: Primary: 65 | Full: 65-113
- Temperature (--temperature): Primary: 31 | Full: 31-167
- Token: Primary: 40 | Full: 40-166
- Top-k (--top_k): Primary: 31 | Full: 31-166
- Validation: Primary: 74 | Full: 74-166

Action 1: What You Learned

- You learned how to use the keyword index as a rapid route back to concepts you need before teaching or revision.
- You learned how index references and glossary entries work together to connect terms to practical chapter examples.
- You learned how planned index use reduces last-minute searching and improves lesson preparation quality.

Action 2: Reflect

- Which indexed term did you locate fastest, and what chapter example made it clear?
- Which important term still needs a better cross-reference for your own revision workflow?
- How would you organize three priority terms before a live teaching session?

Action 3: Do This Next

- Build a "top ten before class" keyword list using the index and note the page for each.

- With a peer, compare your top-ten lists and identify which missing terms should be added for stronger preparation.